



Chapter 13

MySQL を操作する

PHP を使って MySQL データベースからレコードを取り出したり、追加、更新したりする方法を解説します。基本的な SQL の書き方と実行の方法を学び、フォーム入力から MySQL を操作する例やトランザクション処理を使って安全にリレーショナルデータベースを操作する例も示します。

-
- Section 13-1 データベースユーザを追加する
 - Section 13-2 データベースからレコードを取り出す
 - Section 13-3 レコードの抽出、更新、挿入、削除
 - Section 13-4 フォーム入力から MySQL を利用する
 - Section 13-5 リレーショナルデータベースのレコードを取り出す
 - Section 13-6 トランザクション処理
-

Section 13-1

データベースユーザを追加する

PHP からデータベースを利用する際には、安全のために指定のデータベースだけを操作できるユーザを作ります。この節では前節に引き続いて phpMyAdmin を利用してデータベースユーザを作る手順を説明します。

ユーザアカウントを追加する

初期値で使用している root ユーザは、すべてのデータベースを自在に操作できます。PHP からデータベースを利用する際には、プログラムのバグによる事故を未然に防ぐ目的とセキュリティ対策という観点から、利用するデータベースを操作するために必要十分な権限をもった一般ユーザを追加します。

testdb データベースを利用できるユーザを追加する

Chapter12 で作成した testdb データベースを利用できる一般ユーザを追加する手順を説明します。

1 ユーザアカウントを追加する

ナビゲーションパネルのホームアイコン  (または phpMyAdmin ロゴ) をクリックしてメインページを表示し、ユーザアカウントタブをクリックしてユーザアカウント概略を表示します。現在登録済みのユーザリストの下にある「ユーザアカウントを追加する」をクリックして新規ユーザを追加します。

1. ホームをクリックしてメインページを表示します

2. ユーザアカウントを開きます



ユーザアカウント概要

localhostから任意のユーザに接続を許可するユーザが存在します。これにより、そのアカウントのホスト部分で任意ホスト (%) からの接続が許可されている場合、他のユーザは接続できなくなります。

ユーザ名	ホスト名	パスワード	グローバル権限	ユーザグループ	権限委譲	操作
☐ すべて	%	いいえ	USAGE		いいえ	権限を編集 エクスポート
☐ すべて	localhost	いいえ	USAGE		いいえ	権限を編集 エクスポート
☐ pma	localhost	いいえ	USAGE		いいえ	権限を編集 エクスポート
☐ root	127.0.0.1	いいえ	ALL PRIVILEGES		はい	権限を編集 エクスポート
☐ root	::1	いいえ	ALL PRIVILEGES		はい	権限を編集 エクスポート
☐ root	localhost	いいえ	ALL PRIVILEGES		はい	権限を編集 エクスポート

↑ ☐ すべてチェックする チェックしたものを: エクスポート

新規作成

ユーザアカウントを追加する

3. 「ユーザアカウントを追加する」をクリックします

2 ログイン情報や権限を設定する

ユーザアカウントを追加する画面が表示されたならば、ログイン情報を入力します。「ホスト名」では「ローカル」を選択します。すると右のフィールドに localhost と入ります。「パスワードを生成する」にある「生成する」ボタンはパスワードを自動生成するためのボタンです。

ログイン情報	メニューの選択	(入力例)
ユーザ名	テキスト入力項目の値を利用する	testuser
ホスト名	ローカル	localhost
パスワード	テキスト入力項目の値を利用する	pw4testuser
再入力		pw4testuser

ログイン情報を入力したならば、グローバル権限などの他のチェックボックスは1つもチェックせずにそのまま下までスクロールして「実行」ボタンをクリックします。

The screenshot shows the 'phpMyAdmin' interface for adding a new user account. The 'Login Information' section is highlighted with a blue box and labeled '1. 入力します'. The 'Global Privileges' section is shown with all checkboxes unselected, labeled '1つもチェックしません'. The 'Execute' button at the bottom is labeled '2. 「実行」をクリックします'.

利用できるデータベースを指定する

ユーザアカウントの追加が成功したならば、testuser ユーザ (testuser@localhost) がユーザアカウント概略のリストに追加されます。ここでグローバル権限が USAGE になっているかどうかを確認してください。ALL PRIVILEGES は root 権限です。続いて testuser ユーザが利用できるデータベースを指定します。

1 追加ユーザの権限を指定する

ユーザアカウント概略のリストで testuser ユーザの行の「権限を編集」をクリックします。



USAGE かどうか確認してください

testuser の「権限を編集」をクリックします

2 利用できるデータベースを指定する

testuser ユーザの権限設定画面になるので、上に並んだボタンから「データベース」をクリックします。「データベース固有の権限」の欄にあるプルダウンメニューから「testdb」データベースを選択し、実行します。

1. ユーザアカウントが開きます

2. 「データベース」をクリックします



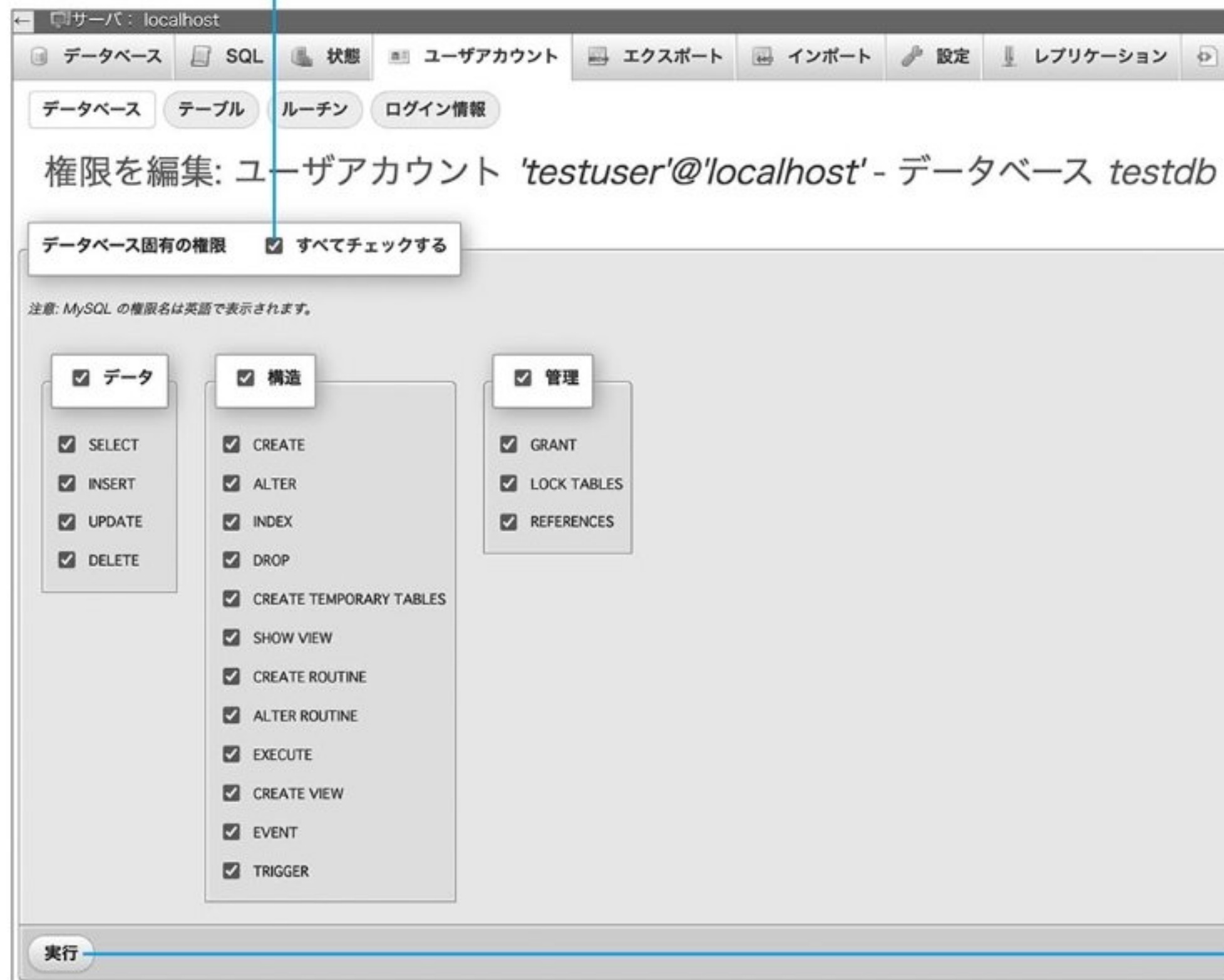
3. testdb データベースを選択します

4. 「実行」をクリックします

3 データベース固有の権限をすべてチェックする

続いて「データベース固有の権限」を設定する画面になるので、「すべてチェックする」をチェックして実行ボタンをクリックします。

1. 「すべてをチェックする」をチェックします



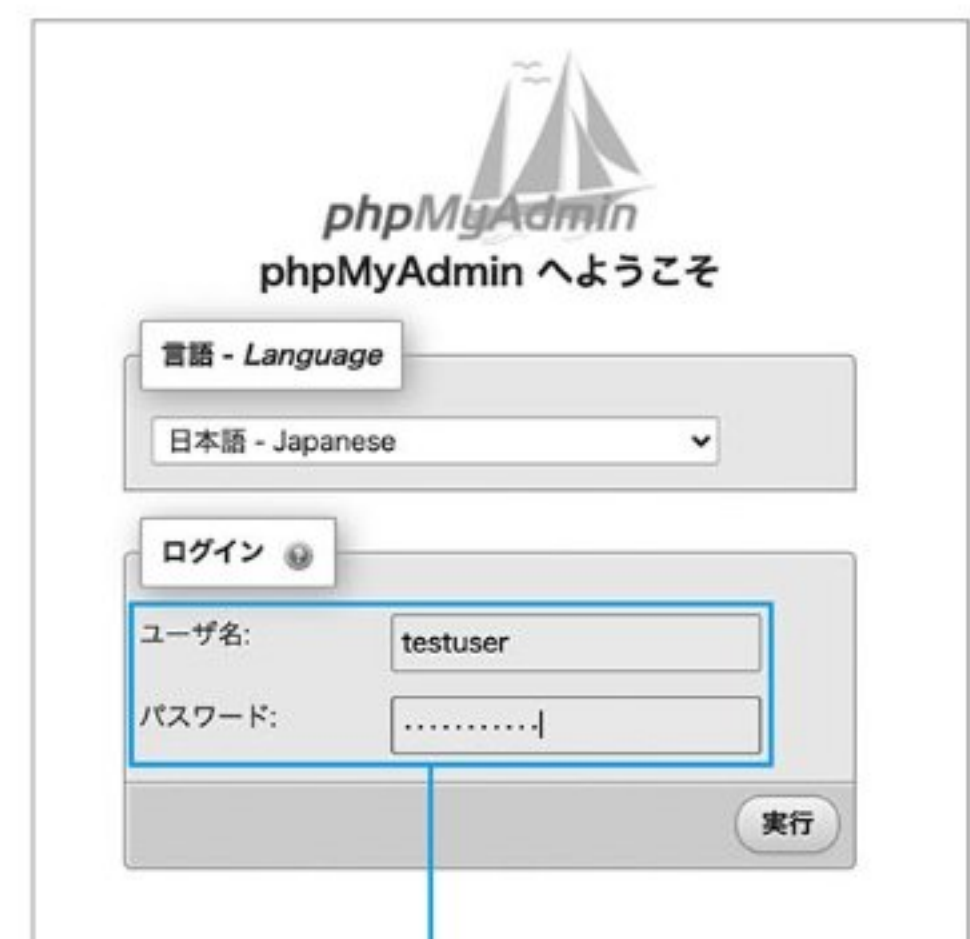
2. 「実行」をクリックします

ログイン画面が出るように設定ファイルを書き替える

phpMyAdmin を起動すると初期状態では root で自動的にログインします。この状態はセキュリティ的にも危険です。また、追加したユーザアカウントでログインし直すこともできません。root パスワードはユーザアカウントの設定画面で変更できますが、先にログイン画面が表示される設定に変更しておかないと自動ログインでパスワードエラーになり、そのままでは phpMyAdmin を起動できなくなるので注意が必要です。

設定ファイル config.inc.php の修正箇所

phpMyAdmin のログイン画面が表示されるようにするために修正する設定ファイルは config.inc.php です。後述するように利用している環境によって config.inc.php を開く方法が異なりますが、



ユーザアカウントとパスワードでログインできるようにします

どの場合も修正箇所は同じです。

config.inc.php を開いたならば、Authentication の設定項目にある `auth_type` の値を 'config' から 'cookie' に変更し、`user` の値を 'root' から空の '' にします。もし、`password` の値が空でなかった場合は、それが現在の root パスワードなので必ず覚えておいてください（初期値で 'root' になっている可能性があります）。その上で `password` の値を空の '' に変更します（root パスワードの設定 P.536）。

変更前 (phpmyadmin/config.inc.php) : root で自動ログインする設定

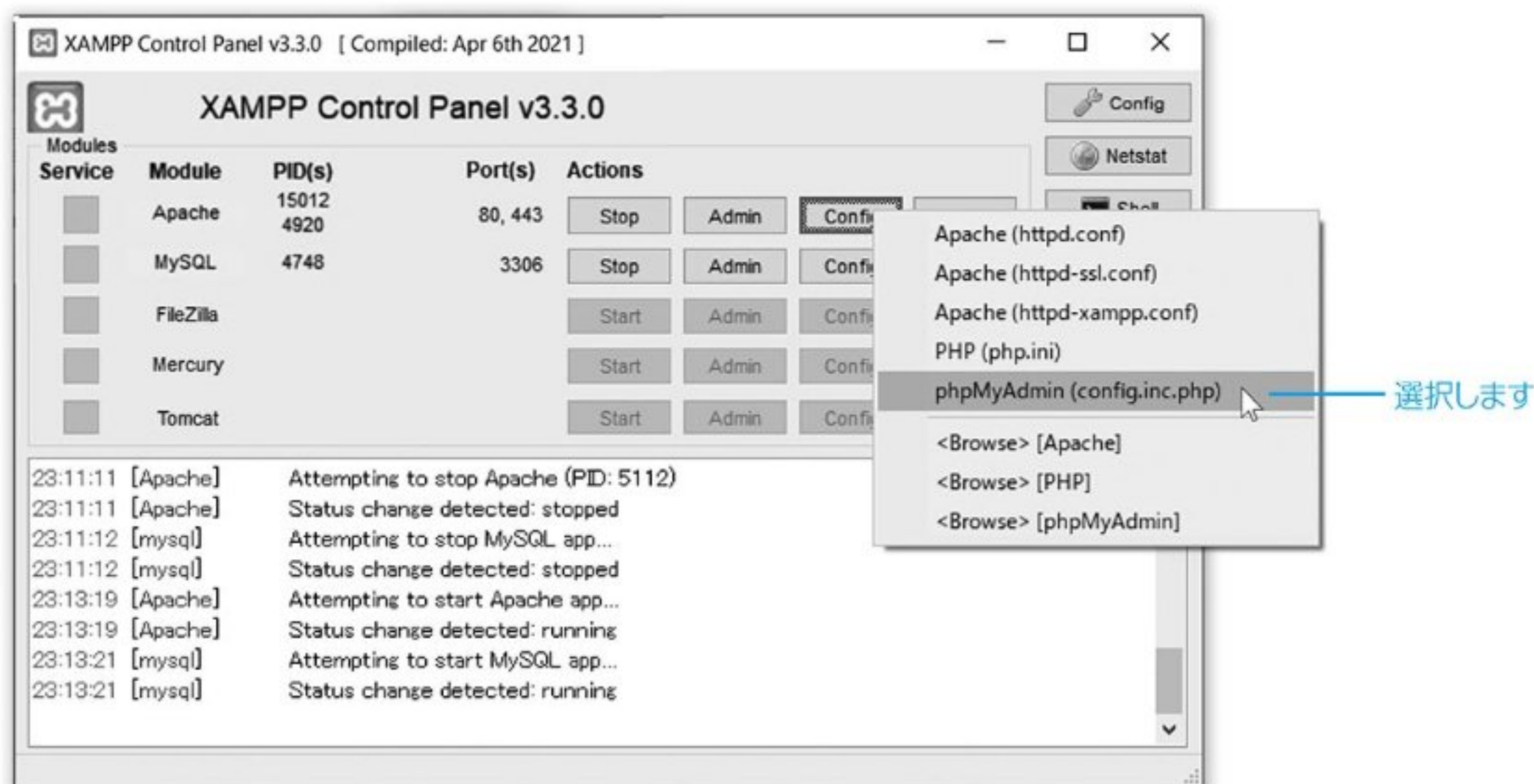
```
27: /* Authentication type */
28:$cfg['Servers'][$i]['auth_type'] = 'config';
29:$cfg['Servers'][$i]['user'] = 'root';
30:$cfg['Servers'][$i]['password'] = ''; —— 空でなかった場合はパスワードを覚えておいてください!
```

変更後 (phpmyadmin/config.inc.php) : ログイン画面が表示される設定

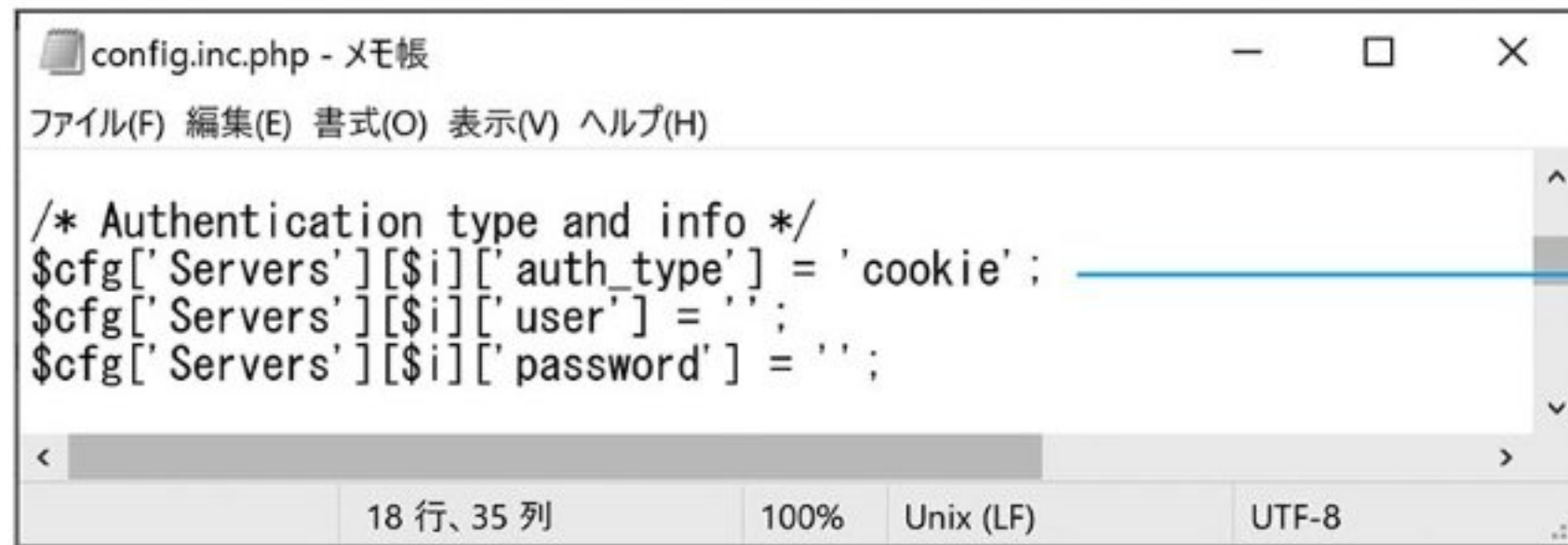
```
27: /* Authentication type */
28:$cfg['Servers'][$i]['auth_type'] = 'cookie';
29:$cfg['Servers'][$i]['user'] = '';
30:$cfg['Servers'][$i]['password'] = '';
```

Windows 版 XAMPP の config.inc.php を開く

Windows 版の XAMPP で config.inc.php を書き替えるには、XAMPP コントロールパネルの Apache の Config ボタンをクリックすると表示されるメニューから「phpMyAdmin (config.inc.php)」を選択します。



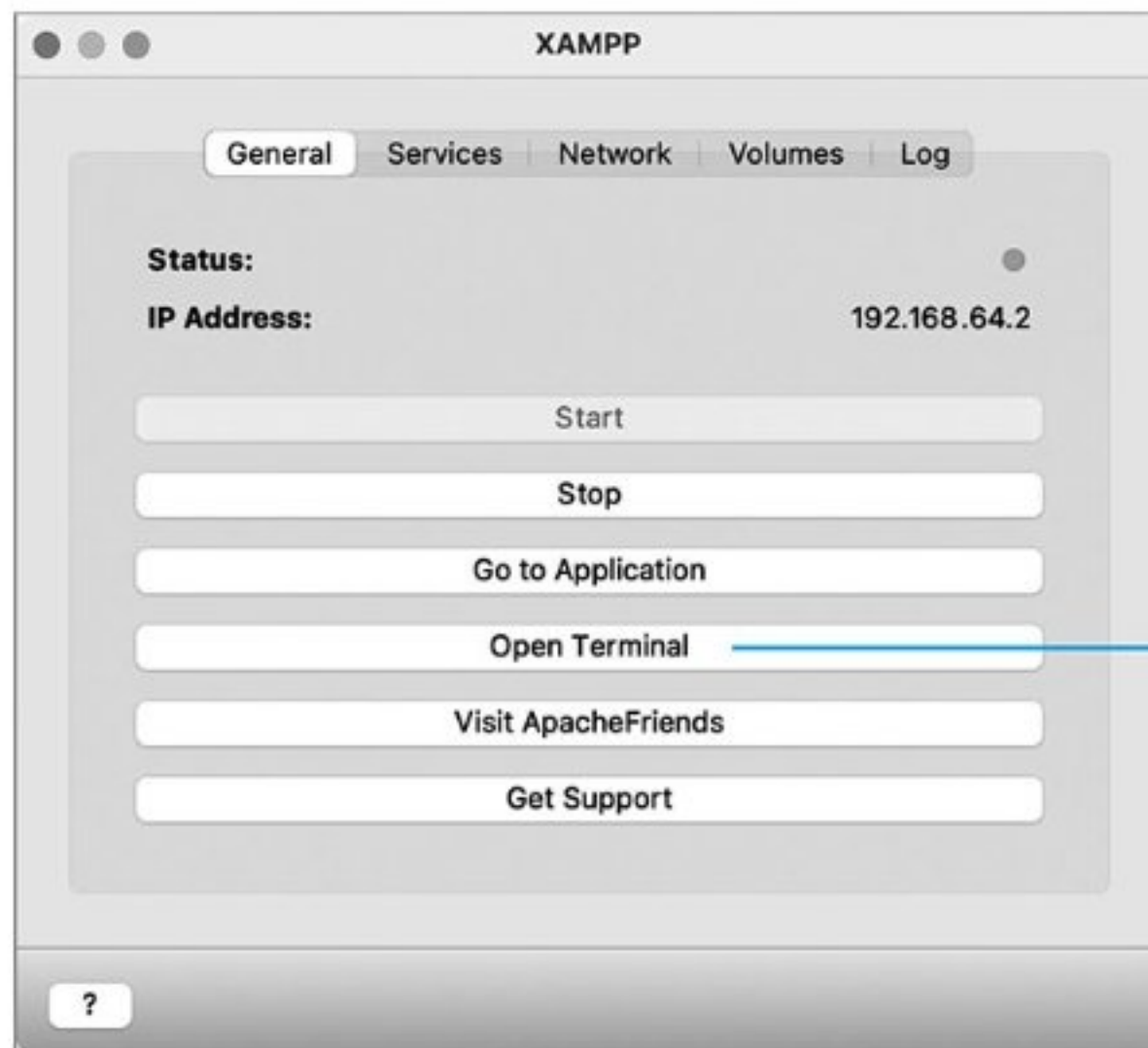
すると、メモ帳アプリで config.inc.php が開くので、先に書いたように「Authentication type and info」にある設定を書き替えて保存します。



phpmyadmin/config.inc.php が開くので、設定を書き替えて保存します

macOS VM 版 XAMPP の config.inc.php を開く

macOS 版の XAMPP で config.inc.php を書き替えるには、XAMPP コントロールパネルでサーバをスタートした状態から「Open Terminal」をクリックします。すると Linux debian の環境がターミナルで開きます。



クリックします

nano エディタをインストールする

まず、Debian のパッケージ管理システムを最新バージョンに更新します。更新が終わったならば、続いて config.inc.php を編集するための nano エディタをインストールします。ターミナルには次のように入力します。1 行目の update コードを入力すると即座に実行されて結果が出力されて止まるので、続けて 2 行目の install コードを実行してください。

Terminal パッケージ管理システムアップデートし、nano エディタをインストールする

```
01: root@debian:~# apt-get update
02: root@debian:~# apt-get install nano
```



```

yoshiyuki — ssh -i ~/.bitnami/stackman/machines/xampp/ssh/id_rsa -o StrictHostKey...
Linux debian 4.19.0-13-amd64 #1 SMP Debian 4.19.160-2 (2020-11-28) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr  8 08:35:00 2021 from 192.168.64.1
root@debian:~# apt-get update
Get:1 http://security.debian.org buster/updates InRelease [65.4 kB]
Hit:2 http://deb.debian.org/debian buster InRelease
Get:3 http://security.debian.org buster/updates/main amd64 Packages [272 kB]
Get:4 http://security.debian.org buster/updates/main Translation-en [146 kB]
Fetched 483 kB in 1s (433 kB/s)
Reading package lists... Done
root@debian:~# apt-get install nano
Reading package lists... Done
Building dependency tree
Reading state information... Done
nano is already the newest version (3.2-3).
0 upgraded, 0 newly installed, 0 to remove and 18 not upgraded.
root@debian:~#

```

パッケージ管理システムを
最新に更新します

nano エディタをインス
トールします

nano エディタで config.inc.php を開く

nano エディタがインストールされたならば、nano エディタを使って config.inc.php を開きます。ターミナルには次のように入力します。

Terminal ターミナルでタイプして config.inc.php を nano エディタで開く

```
01: root@debian:~# nano ../opt/lampp/phpmyadmin/config.inc.php
```

```

yoshiyuki — ssh -i ~/.bitnami/stackman/machines/xampp/ssh/id_rsa -o StrictHostKey...
Linux debian 4.19.0-13-amd64 #1 SMP Debian 4.19.160-2 (2020-11-28) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr  8 08:35:00 2021 from 192.168.64.1
root@debian:~# apt-get update
Get:1 http://security.debian.org buster/updates InRelease [65.4 kB]
Hit:2 http://deb.debian.org/debian buster InRelease
Get:3 http://security.debian.org buster/updates/main amd64 Packages [272 kB]
Get:4 http://security.debian.org buster/updates/main Translation-en [146 kB]
Fetched 483 kB in 1s (433 kB/s)
Reading package lists... Done
root@debian:~# apt-get install nano
Reading package lists... Done
Building dependency tree
Reading state information... Done
nano is already the newest version (3.2-3).
0 upgraded, 0 newly installed, 0 to remove and 18 not upgraded.
root@debian:~# nano ../opt/lampp/phpmyadmin/config.inc.php

```

nano エディタで config.
inc.php を開きます

config.inc.php が開いたならば、矢印キーを使って変更する箇所までカーソルを移動させて書き替えます。該当箇所をすべて書き替えたならば、control + Xに続いてYキーを押して保存します。returnキーでnanoエディタが終了し、ターミナルの元の表示に戻ります。(変更箇所 P.530)

```

yoshiyuki — ssh -i ~/.bitnami/stackman/machines/xampp/ssh/id_rsa -o StrictHostKey...
GNU nano 3.2      ../opt/lampp/phpmyadmin/config.inc.php      Modified

$i = 0;

/**
 * First server
 */
$i++;

/* Authentication type */
$cfg['Servers'][$i]['auth_type'] = 'cookie';
$cfg['Servers'][$i]['user'] = '';
$cfg['Servers'][$i]['password'] = '';
/* Server parameters */
// $cfg['Servers'][$i]['host'] = 'localhost';
$cfg['Servers'][$i]['compress'] = false;
$cfg['Servers'][$i]['AllowNoPassword'] = true;

/**
 * phpMyAdmin configuration storage settings.
 */

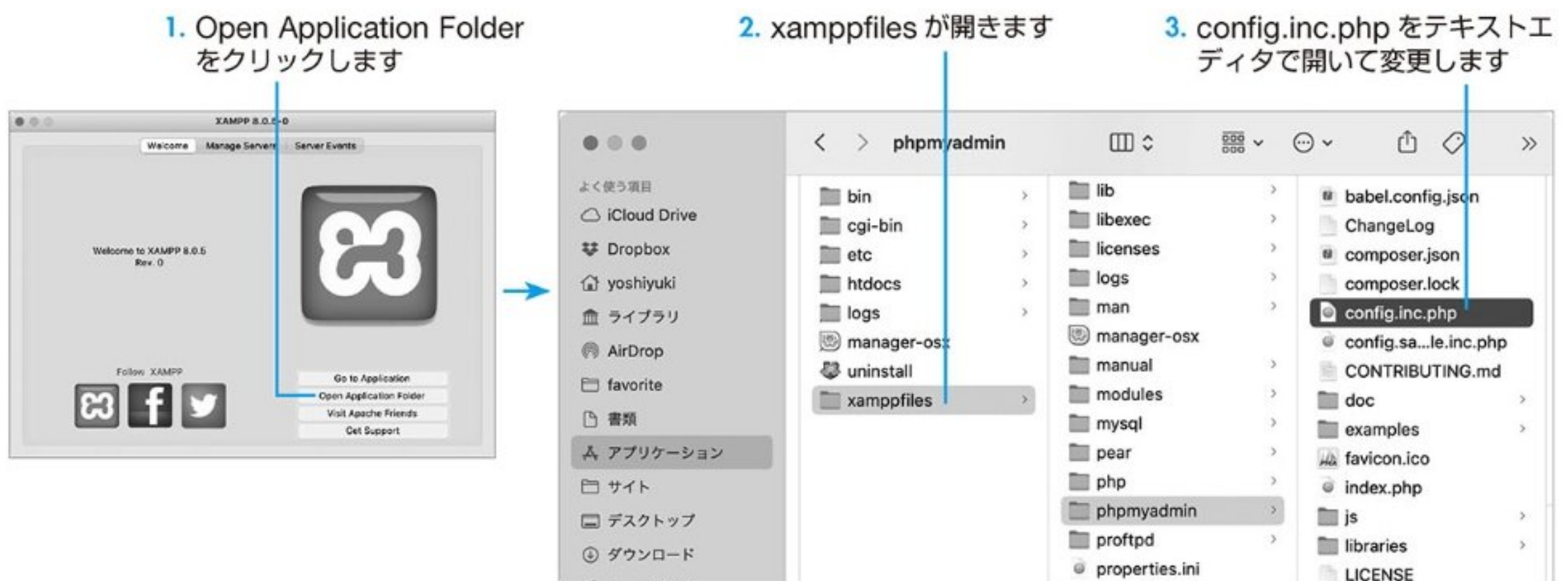
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File  ^_ Replace   ^U Uncut Text ^T To Spell  ^_ Go To Line

```

書き替えて保存します

macOS アプリ版 XAMPP の config.inc.php を開く

XAMPP アプリケーションマネージャーの Open Application Folder をクリックすると Finder で XAMPP > xamppfiles フォルダが開くので、この中にある phpmyadmin フォルダに config.inc.php をテキストエディタで開いて「Authentication type」にある設定を書き替えて保存します。(変更箇所 P.530)



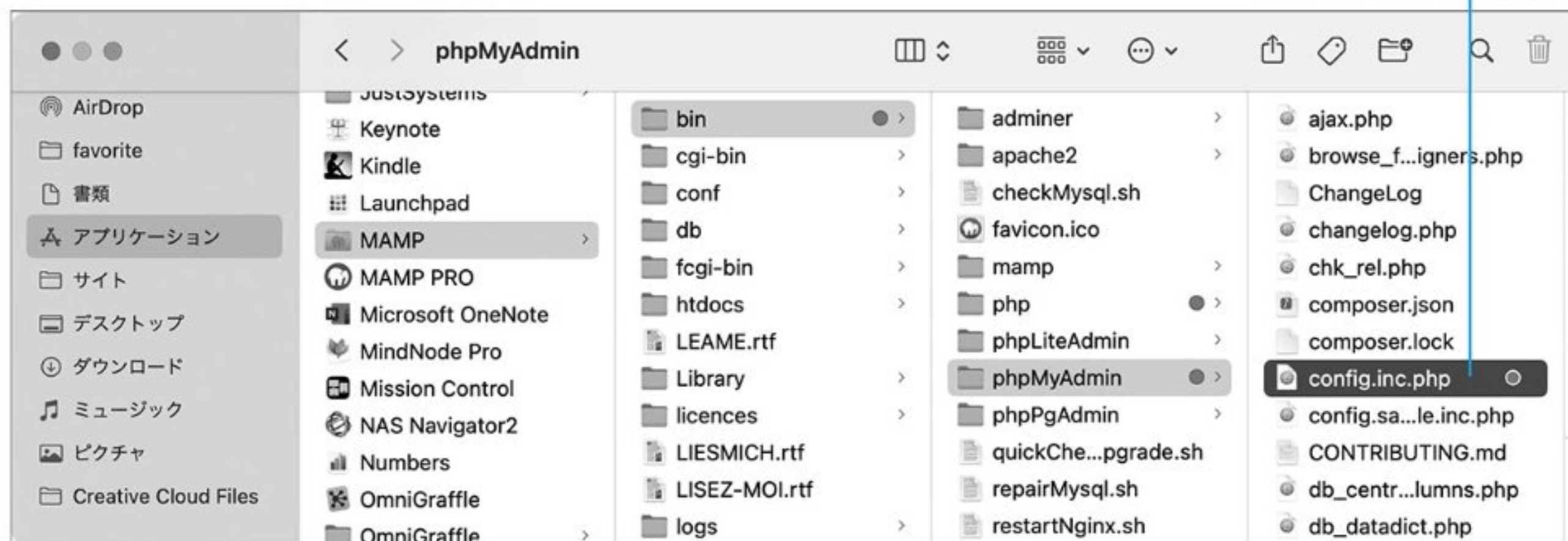
MAMP で config.inc.php を開く

MAMP を利用している場合には、ファインダからテキストエディタで config.inc.php を開いて修正します。config.inc.php はインストールした MAMP の bin>phpMyAdmin フォルダに入っています。テキストエディタには utf-8 に対応したエディタを使ってください。(変更箇所 P.530)

Windows のパス : C:\MAMP\bin\phpMyAdmin\config.inc.php

macOS のパス : /アプリケーション/MAMP/bin/phpMyAdmin/config.inc.php

テキストエディタで開きます



ユーザアカウントでログインする

config.inc.php の書き換えが終わったならば、あらためて phpMyAdmin を起動します。すると図に示すようなログイン画面が表示されるはずです。さきほど作成したユーザアカウント testuser でログインを試してみましょう。

phpMyAdmin を起動するとログイン画面が表示されるようになります



ユーザアカウントでログインすると、ナビゲーションパネルには権限があるデータベースだけが表示されるようになります。データベースを作成したり、ユーザアカウントを追加したりするなどの権限がない機能は表示されません。



操作できる権限があるデータベースだけが表示されます



権限がない機能は利用できません

ログアウトする

ログイン画面からのログインができるように設定を行うと、ホーム  の右の  がログアウトボタンとして機能するようになります。ログアウトボタン  をクリックするとログアウトしてログイン画面に戻ります。

クリックでログアウトできます



root ユーザアカウントでログインする

データベースの作成やユーザアカウントの追加などは root ユーザでログインして行う必要があります。root ユーザでログインするには、変更前の config.inc.php の password の値が空だった場合は root パスワードが設定されていない状態なので、パスワードの入力欄は空のまま実行すればログインできます。もし、変更前の config.inc.php の password の値が空ではなかった場合は、その値をパスワードの入力欄に入れてログインします。

root パスワードを設定する

PHP の学習目的では root パスワードを設定する必要はありませんが、実際の運用にあたっては必ず設定しなければなりません。root パスワードは、root ユーザでログインした状態でユーザアカウントタブを開いて設定します。root パスワードは注意して設定し、取り扱いには十分注意してください。

1. root ユーザでログインし、ユーザアカウントを開きます

The screenshot shows the phpMyAdmin interface for the 'root' user on 'localhost'. The 'Change password' tab is selected. The main content area displays the 'パスワードを変更する' (Change password) section. It includes a 'パスワード' (Password) field, a '再入力' (Re-enter) field, and a 'パスワードハッシュ' (Password hash) dropdown menu. A 'Strength' indicator shows 'Good'. A blue dashed box highlights the password input fields, and a blue arrow points to the '実行' (Execute) button at the bottom.

2. パスワードを設定します

3. クリックします

NOTE

非公開パズフレーズを設定する警告文が出る場合

「設定ファイルに、暗号化 (blowfish_secret) 用の非公開パズフレーズの設定を必要とするようになりました。」といった警告文が出る場合は、config.inc.php の次の箇所を書き替えます。指定するパズフレーズは何でも構いません。

php 変更前

```
39: // $cfg['blowfish_secret'] = '';
```

php 変更後 (例)

```
39: $cfg['blowfish_secret'] = 'php8mysecret';
```


Section 13-2

データベースからレコードを取り出す

いよいよ PHP を使って MySQL データベースからデータを取り出します。MySQL には PDO クラスで接続し、SQL 文を PHP から MySQL に送って操作します。この節ではデータベースへの接続とレコードデータの取り出し方の基本を説明します。

データベースを準備する

Section12-2 で作成したデータベース testdb に、前節で追加したデータベースユーザ testuser で接続します。testdb は取り出し条件を組み合わせた例を試すにはカラム数が少ないので、性別(sex)カラムを追加しましょう。レコードもあと3人分ほど追加したいと思います。性別カラムとレコードを追加したテーブルの内容は右の表のとおりです。次節以降もこのデータベースを使って、PHP からの接続とレコードの値を取り出す方法を説明します。

ID(id)	名前 (name)	年齢 (age)	性別 (sex)
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女
4	高峯信夫	23	男
5	新倉建雄	51	男
6	青木由香里	32	女
7	佐々木伸吾	28	男
8	井上珠理	27	女

1 性別カラムを追加する

member テーブルを選択して構造タブを開きます。「1 個のカラムを追加する age の後へ」の右の「実行」ボタンをクリックしてカラムを追加します。

2. 構造タブを開きます



1. member テーブルを選択します

3. クリックしてカラムを追加します

2 性別カラムの設定を行う

カラムの設定が表示されるので、名前「sex」、タイプ「VARCHAR」、長さ／値「2」、デフォルト値ではユーザ定義を選択して「男」にします。以上を入力したならば「保存する」ボタンをクリックします。テーブルの構造に sex カラムが追加されます。

1. カラムの属性を指定します



2. クリックして保存します



3. sex カラムが追加されます

3 入力済みのレコードの性別を変更する

表示タブを開いて現在登録済みのレコードを見ると前節で追加した5人のレコードが入っています。いま追加した sex カラムの値が全員初期値の「男」になっているので、塩田香織、雨木さくらの2人の sex の値を「女」に変更します。値の変更はカラムのフィールドをクリックして書き替えることができます。

1. 表示タブを開きます



2. クリックして2人の性別を書き替えます

4 3名のレコードを追加する

挿入タブを開き、3人のレコードを追加します。id は自動入力されるので空のままにしておきます。入力が終わったならば表示タブを開いて追加した3人のレコードを確認してください。

1. 挿入タブを開きます

2. 名前、年齢、性別を入力します。
id は入力しません

3. 続けてもう1人追加するので、「新しいレコードを追加する」を選んでおきます

4. クリックして実行します

5. 3人を追加したら表示タブを開きます

6. 追加した3人のレコードを確認します

	id	name	age	sex
☐	1	佐藤一郎	32	男
☐	2	堀田香織	26	女
☐	3	雨木さくら	38	女
☐	4	高峯信夫	23	男
☐	5	新倉健雄	51	男
☐	6	青木由香里	32	女
☐	7	佐々木伸吾	28	男
☐	8	井上珠理	27	女

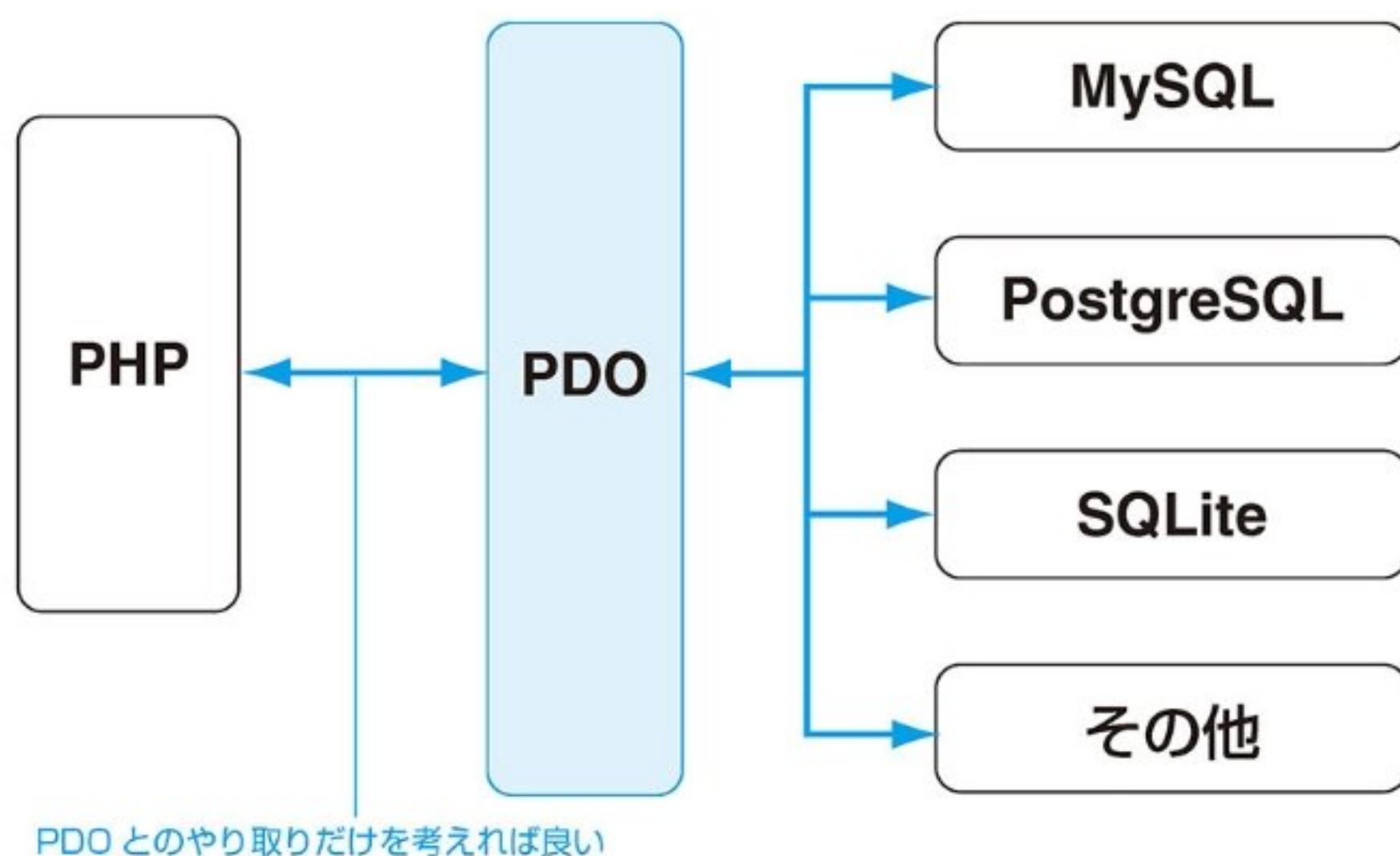
NOTE**性別をプルダウンメニューで入力できるようにしたい**

性別をプルダウンメニューで選択できるようにするには、新規に性別テーブルを作りリレーションするようにします。商品データベースのブランドを参考にしてください。(P.513、P.515、P.517)

データベースに接続する

利用するデータベースの準備ができたならば、いよいよデータベースに接続します。PHP からデータベースに接続するには PDO (PHP Data Objects) を利用します。データベースには MySQL、PostgreSQL、SQLite などいろいろな種類があり、データベースごとに接続方法や操作方法などが違ってきます。PDO はそれらの違いを吸収してくれる機能 (抽象化レイヤ) で、PDO を使うことでデータベースの違いを意識せずに PHP コードを書くことができます。

データベースの相違点を PDO が解決してくれる



次に示すのは PHP から MySQL データベースの testdb に接続するだけのコードです。接続に成功したならば「データベース testdb に接続しました。」と出力されます。接続できなかったならば「エラーがありました。」に続いて、発生したエラーメッセージが出力されます。

php PHP から MySQL データベースに接続する

«sample» connect/PDO_testdb.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>PDO でデータベースに接続する </title>
06: <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:     <?php
11:         // データベースユーザ
12:         $user = 'testuser';
13:         $password = 'pw4testuser';
14:         // 利用するデータベース
15:         $dbName = 'testdb';
16:         // MySQL サーバ
17:         $host = 'localhost:3306';
18:         // MySQL の DSN 文字列
19:         $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
20:
21:         //MySQL データベースに接続する
22:         try {
23:             $pdo = new PDO($dsn, $user, $password);
24:             // プリペアドステートメントのエミュレーションを無効にする
25:             $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
26:             // 例外がスローされる設定にする
27:             $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
28:             echo " データベース {$dbName} に接続しました。 ";
29:             // 接続を解除する
30:             $pdo = NULL;
31:         } catch (Exception $e) {
32:             echo '<span class="error">エラーがありました。</span><br>';
33:             echo $e->getMessage();
34:             exit();
35:         }
36:     ?>
37: </div>
38: </body>
39: </html>

```

DSN 文字列を作ります

データベースに接続します

接続に失敗したら、例外処理が実行されます

データベースの接続に成功した場合



データベースの接続に失敗した場合



PDO クラスを介してデータベースに接続する

まず最初にデータベースに接続するユーザ名、パスワード、さらにデータベース名、ホストを指定して DSN (Data Source Name) の文字列を作ります。データベースユーザには root ユーザではなく、前節で追加した一般ユーザを使います (P.526)。\$host に代入する 'localhost:3306' の 3306 は MySQL のポート番号です (MySQL のポート番号を確認する 次ページ)。

php データベースユーザと DSN を用意する

«sample» connect/PDO_testdb.php

```

11: // データベースユーザ
12: $user = 'testuser';
13: $password = 'pw4testuser';
14: // 利用するデータベース
15: $dbName = 'testdb';
16: // MySQL サーバ
17: $host = 'localhost:3306';
18: // MySQL の DSN 文字列
19: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";

```

安全のために root ユーザは使いません

3306 は MySQL のポート番号です

new PDO(\$dsn, \$user, \$password) で PDO クラスのインスタンス \$pdo を作るかたちで、DSN で指定したデータベースにデータベースユーザで接続します。接続する際には try ~ catch の例外処理の構文を使い、接続に失敗した場合はスローされる例外オブジェクトをキャッチして対応します。続けて setAttribute() を使って、プリペアドステートメントのエミュレーションを無効にする設定とエラーモードの設定も行います。エラーモードには例外をスローする設定を指定します。

php データベースに接続する

«sample» connect/PDO_testdb.php

```

21: //MySQL データベースに接続する
22: try {
23:     $pdo = new PDO($dsn, $user, $password);
24:     // プリペアドステートメントのエミュレーションを無効にする
25:     $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
26:     // 例外がスローされる設定にする
27:     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
28:     ... (接続したデータベースを操作します)
29:
30: } catch (Exception $e) {
31:     echo '<span class="error">エラーがありました。</span><br>';
32:     echo $e->getMessage();
33:     exit();
34: }

```

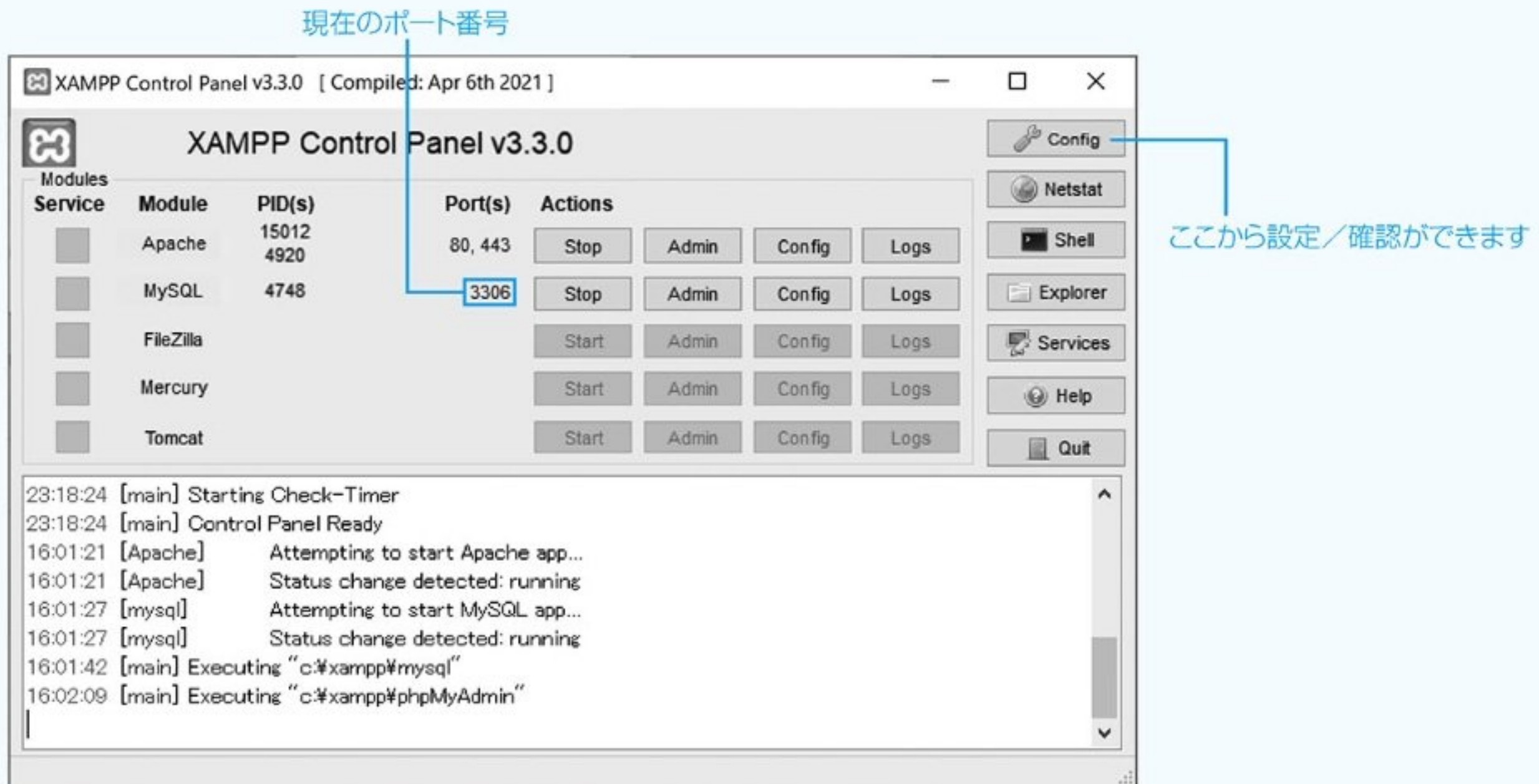
以上のコードでデータベースへの接続を確かめることができます。接続の解除は自動的に行われるので何もしなくても構いませんが、\$pdo = NULL で PDO インスタンスを破棄すれば接続は解除されます。

NOTE

MySQL のポート番号を確認する

XAMPP の Windows 版で MySQL のポート番号を確認する方法

XAMPP を起動すると表示されるコントロールパネルにポート番号が表示されています。右上の Config ボタンから開く XAMPP の設定の Service and Port Settings で設定と確認ができます (P.21)。



macOS VM 版 XAMPP で MySQL のポート番号を確認する方法

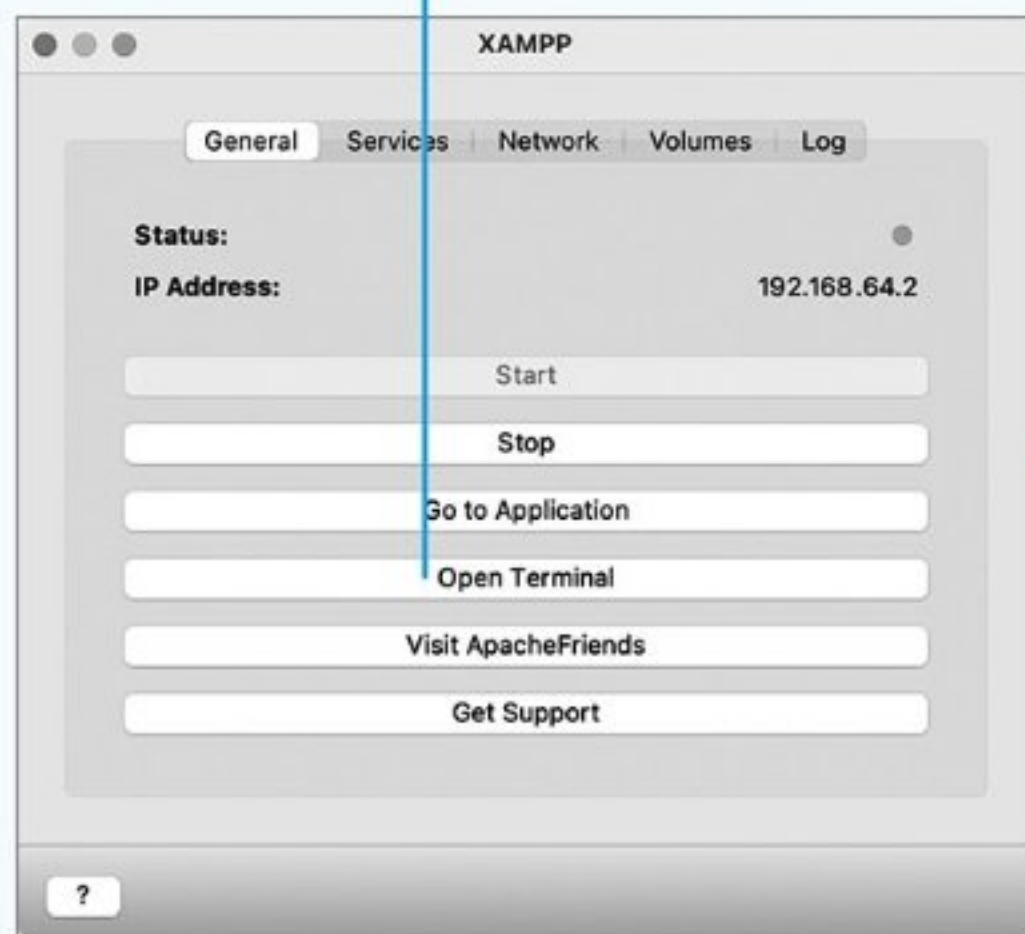
XAMPP の MySQL サーバを起動しておき、コントロールパネルの Open Terminal ボタンをクリックしてターミナルを開きます。ターミナルから MySQL にログインしてポート番号を調べます。

ターミナルから入力するコマンドは次のとおりです。次の例では root ユーザでログインしていますが、一般のユーザアカウントでも構いません。ここでの root ユーザ、ユーザアカウントとは phpMyAdmin で設定しているユーザアカウント、すなわち MySQL のユーザアカウントのことです。

Terminal MySQL にログインしてポート番号を調べる

```
01: root@debian:~# mysql -u root -p      ——— root でログインします
02: Enter password: ****                ——— root パスワードを入力します
03: MariaDB [(none)]> show variables like 'port';
04: ... ここにポート番号が出力されます
05: MariaDB [(none)]> exit              ——— MySQL をログアウトします
```


1. クリックしてターミナルを開きます



2. MySQL にログインします

```
yoshiyuki ~ ssh -i ~/.bitnami/stackman/machines/xampp/ssh/id_rsa -o StrictHostKey...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Apr 11 20:29:41 2021 from 192.168.64.1
root@debian:~# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.4.18-MariaDB Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show variables like 'port';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| port          | 3306  |
+-----+-----+
1 row in set (0.002 sec)

MariaDB [(none)]> exit
Bye
root@debian:~#
```

4. ポート番号が表示されます

3. ポート番号を調べます

macOS アプリ版 XAMPP で MySQL のポート番号を確認する方法

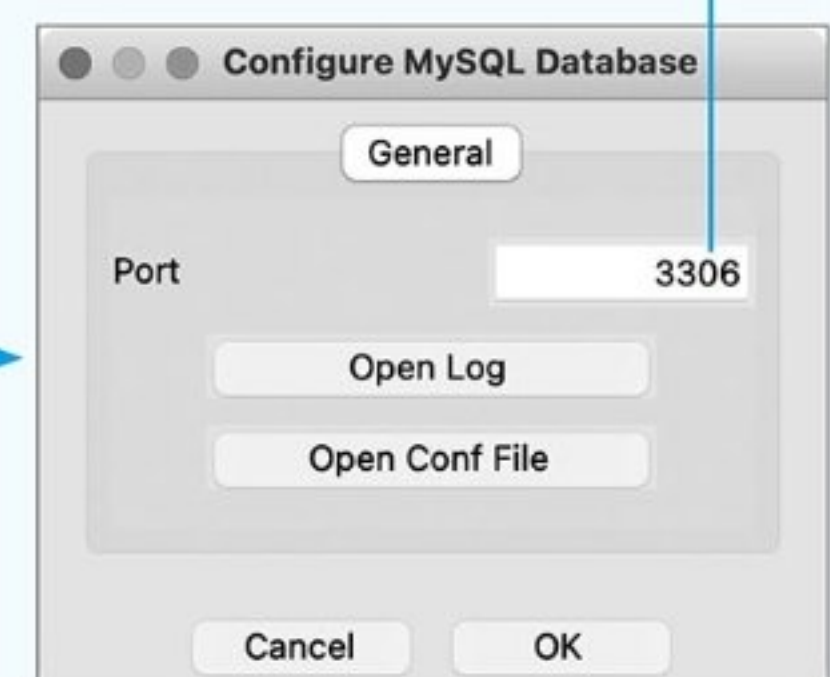
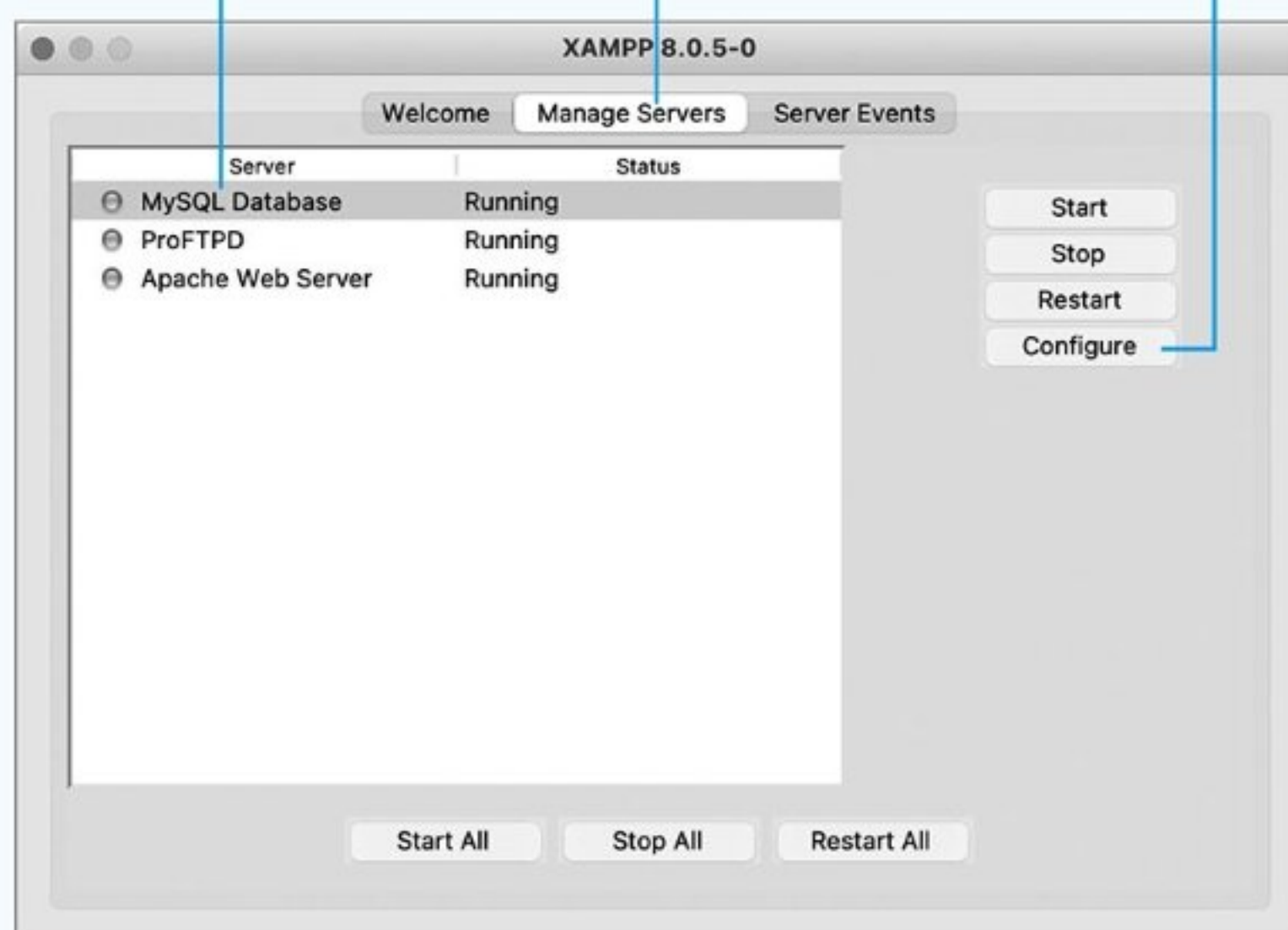
XAMPP アプリケーションマネージャーの Manage Servers を開き、リストの MySQL Database を選択します。Configure ボタンをクリックすると MySQL のポート番号を設定/確認するウィンドウが表示されます。

1. Manage Servers を開きます

2. MySQL Database を選択します

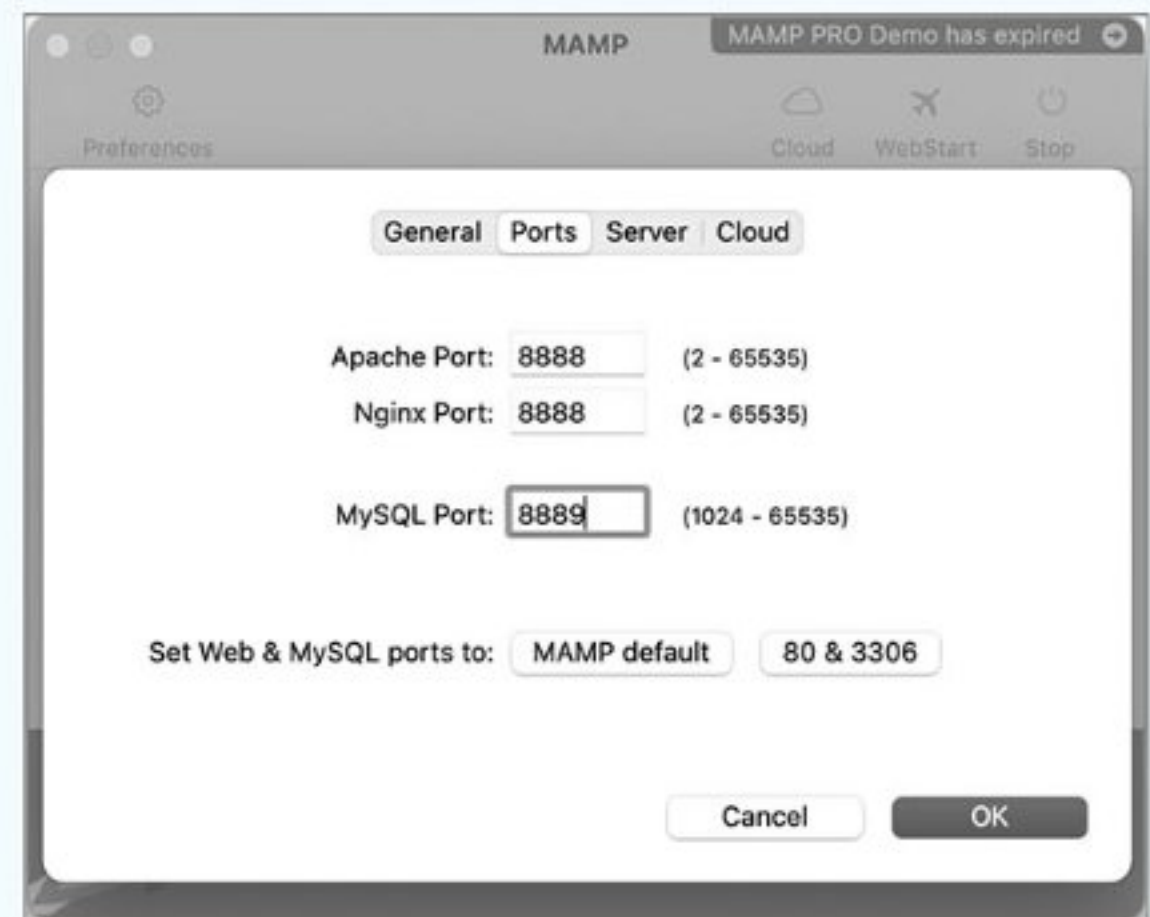
3. クリックします

4. MySQL のポート番号が表示されます



MAMP で MySQL のポート番号を確認する方法

MAMP を起動すると表示されるコントロールパネルの Preferences ボタンをクリックして設定画面を表示し、Ports タグを開くと MySQL のポート番号を確認／設定ができます。



レコードデータを取り出す

データベースに接続したならば、次にレコードデータを取り出します。MySQL データベースを操作するには、MySQL を操作するプログラム言語の SQL 文を使います。SQL 文の実行では、いったんプリペAREDステートメントに変換して実行する方式を使います。

次のコードを実行すると、member テーブルにあるレコードをすべて取り出して表にして表示します。全体の流れとしては、大まかに次のようになります。

1. testdb データベースに接続する。
2. member テーブルからレコードを取り出す SQL 文（プリペAREDステートメント）を用意する
3. SQL 文を実行する
4. 取り出したレコードを HTML のテーブルで表示する

レコードを取り出す (すべて)
+
-
□
×

←
→
↺
localhost:8888/php_nc
☆
...

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女
4	高峯信夫	23	男
5	新倉建雄	51	男
6	青木由香里	32	女
7	佐々木伸吾	28	男
8	井上珠理	27	女

testdb の member テーブルから
すべてのレコードが取り出されます

php member テーブルのレコードをすべて取り出す

«sample» select/all.php

```

01: <?php
02: require_once("../../lib/util.php");
03: // データベースユーザ
04: $user = 'testuser';
05: $password = 'pw4testuser';
06: // 利用するデータベース
07: $dbName = 'testdb';
08: // MySQL サーバ
09: $host = 'localhost:8889';
10: // MySQL の DSN 文字列
11: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
12: ?>
13:
14: <!DOCTYPE html>
15: <html lang="ja">
16: <head>
17: <meta charset="utf-8">
18: <title>レコードを取り出す (AND) </title>
19: <link href="../../css/style.css" rel="stylesheet">
20: <!-- テーブル用のスタイルシート -->
21: <link href="../../css/tablestyle.css" rel="stylesheet">
22: </head>
23: <body>
24: <div>
25: <?php
26: //MySQL データベースに接続する
27: try {
28:     $pdo = new PDO($dsn, $user, $password);
29:     // プリペアドステートメントのエミュレーションを無効にする
30:     $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
31:     // 例外がスローされる設定にする
32:     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
33:     echo " データベース {$dbName} に接続しました。", "<br>";
34:     // SQL 文を作る (全レコード)
35:     $sql = "SELECT * FROM member";
36:     // プリペアドステートメントを作る
37:     $stm = $pdo->prepare($sql);
38:     // SQL 文を実行する
39:     $stm->execute();
40:     // 結果の取得 (連想配列で受け取る)
41:     $result = $stm->fetchAll(PDO::FETCH_ASSOC);
42:     // テーブルのタイトル行
43:     echo "<table>";
44:     echo "<thead><tr>";
45:     echo "<th>", "ID", "</th>";
46:     echo "<th>", "名前", "</th>";
47:     echo "<th>", "年齢", "</th>";
48:     echo "<th>", "性別", "</th>";
49:     echo "</tr></thead>";
50:     // 値を取り出して行に表示する
51:     echo "<tbody>";
52:     foreach ($result as $row){
53:         // 1行ずつテーブルに入れる
54:         echo "<tr>";
55:         echo "<td>", es($row['id']), "</td>";
56:         echo "<td>", es($row['name']), "</td>";
57:         echo "<td>", es($row['age']), "</td>";

```

接続パラメータを準備します

データベースに接続します

SQL 文を実行して、レコードを取り出します
(次節ではこの範囲の SQL 文を変更していきます)

取り出したレコードの値を表示します


```

58:         echo "<td>", es($row['sex']), "</td>";
59:         echo "</tr>";
60:     }
61:     echo "</tbody>";
62:     echo "</table>";
63: } catch (Exception $e) {
64:     echo '<span class="error">エラーがありました。</span><br>';
65:     echo $e->getMessage();
66:     exit();
67: }
68: ?>
69: </div>
70: </body>
71: </html>

```

NOTE**DDL、DML、DCL**

SQL 文はデータベース定義文 (DDL : Data Definition Language)、データ操作文 (DML : Data Manipulation Language)、データ制御文 (DCL : Data Control Language) の3種類に大きく分けることができます。レコードの抽出、追加、削除などを行う SELECT、INSERT、UPDATE、DELETE といった命令はデータ操作文 DML に含まれます。

SQL 文のプリペアドステートメントを作る

データベースからレコードを取り出す SQL 文は、SELECT 命令を使って次のように書きます。

書式 SELECT 命令

SELECT カラム **FROM** テーブル **WHERE** 条件 **LIMIT** 開始位置, 行数

カラムにワイルドカードの * を指定するとすべてのカラムを取り出します。WHERE 以下を省略すると、条件なしですべてのレコードが取り出す対象になります。次の SELECT 文は、member テーブルにあるすべてのレコードのすべてのカラムの値を取り出す命令文になります。SQL 文の全体を " " でくくって文字列にして \$sql に代入します。

php member テーブルのレコードをすべて取り出す SQL 文を作る

«sample» select/all.php

```

34: // SQL 文を作る (全レコード)
35: $sql = "SELECT * FROM member";

```

SQL 文の全体を " " でくくって代入します
すべてのカラム member テーブル

プリペアドステートメントを作って実行する

この SQL 文を \$pdo->prepare(\$sql) でプリペアドステートメント \$stm に変換し、\$stm->execute() で実行します。SQL 文の実行では、SQL 文の構造解析、コンパイル、最適化が行われます。SQL 文をプリペア

ドステートメントにしておくと、同じ SQL 文を繰り返し実行する場合に最初の 1 回だけで処理が完了します。また、次節で詳しく取り上げますが、プリペアドステートメントではプレースホルダが使えるという大きな利点があります。

php SQL 文のプリペアドステートメントを作って実行する

«sample» select/all.php

```
36: // プリペアドステートメントを作る
37: $stm = $pdo->prepare($sql);
38: // SQL 文を実行する
39: $stm->execute();
```

結果を受け取って表示する

SQL 文を実行した結果を受け取るには、`fetch()` または `fetchAll()` をあらためて実行します。`fetchAll()` を実行すると、変数 `$result` にすべてのレコードの値を連想配列のかたちで受け取ることができます。引数の `PDO::FETCH_ASSOC` がレコードを連想配列で取り出す指定です。

php 結果を受け取る

«sample» select/all.php

```
40: // 結果の取得（連想配列で受け取る）
41: $result = $stm->fetchAll(PDO::FETCH_ASSOC);
```

変数 `$result` は連想配列なので、次のように `foreach` 文を使って `$row` に 1 レコードずつ順に取り出すことができます。なお、ブラウザには念のために HTML エスケープを行った値を表示します。

php 結果を 1 レコードずつ取り出して表示する

«sample» select/all.php

```
52: foreach ($result as $row){
53:     // 1行ずつテーブルに入れる
54:     echo "<tr>";
55:     echo "<td>", es($row['id']), "</td>";
56:     echo "<td>", es($row['name']), "</td>";
57:     echo "<td>", es($row['age']), "</td>";
58:     echo "<td>", es($row['sex']), "</td>";
59:     echo "</tr>";
60: }
```


取り出すレコード数を指定する

SELECT 命令に LIMIT 句を付けると取り出す開始位置と行数（レコード数）を指定できます。開始位置は省略でき、省略すると先頭から取り出します。先の SQL 文に次のように LIMIT 句を付けると先頭から 3 人だけを取り出します。

php 3 人だけ取り出す

«sample» select/limit.php

```
35: $sql = "SELECT * FROM member LIMIT 3";
```

先頭から3レコードを対象にします

レコードを取り出す (LIMIT × + − □ ×)			
←	↺	/13-2/select/limit.php	📖 ☆ ...
データベースtestdbに接続しました。			
ID	名前	年齢	性別
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女

最初の 3 人のレコードが取り出されます

Section 13-3

レコードの抽出、更新、挿入、削除

前節で MySQL データベースに接続して SQL 文を実行するところまでを試しました。本節ではさらに条件を満たすレコードの抽出、複数の条件での抽出、値のソート、値の更新、レコードの挿入、レコードの削除といった、より具体的なレコード操作を行います。

30 歳以上の女性を選び出す

では手始めに「30 歳以上の女性」を選び出して表示してみましょう。前節の最後に説明した member テーブルからすべてのレコードを取り出す all.php との違いは SQL 文だけなので (P.546:色が敷いてある範囲)、その部分だけを抜き出して説明します。

「30 歳以上の女性」という条件をより具体的にすると「age の値が 30 以上、sex が " 女 " の 2 つの条件を両方満たす」という条件になります。age の値が 30 以上という条件は「age >= 30」の式になります。sex が " 女 " の条件は「sex = ' 女 '」の式になります。この 2 つの条件を「WHERE age >= 30 AND sex = ' 女 '」のように WHERE を付けて AND で連結すると 2 つの条件を両方満たすレコードを探す条件式になります。なお、SQL 文の全体を " " でくくるので ' 女 ' のようにシングルクォーテーションを使います。

php age が 30 以上、sex が " 女 " のレコードを選び出す

«sample» select/and.php

```
34: // SQL 文を作る (30 以上、女性)
35: $sql = "SELECT * FROM member WHERE age >= 30 AND sex = ' 女 '";
36: // プリペアドステートメントを作る
37: $stmt = $pdo->prepare($sql);
38: // SQL 文を実行する
39: $stmt->execute();
```

「30 歳以上」かつ「女性」

レコードを取り出す (すべて) × + − □

localhost:8888/php_nc

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女
4	高峯信夫	23	男
5	新倉建雄	51	男
6	青木由香里	32	女
7	佐々木伸吾	28	男
8	井上珠理	27	女

レコードを取り出す (AND) × + − □

/13-3/select/and.php

データベースtestdbに接続しました。

ID	名前	年齢	性別
3	雨木さくら	38	女
6	青木由香里	32	女

30 歳以上の女性を選び出します

20 代を年齢順に取り出す

20 代を年齢順、つまり「age の値が 20 以上で 30 未満」の条件で「age の値の順」で取り出す SQL 文は次のようになります。値は >、>=、<、<=、= といった比較演算子で大きさを比較できます。値の順は ORDER BY でキーとなるカラムを指定します。並びの昇順、降順は ASC、DESC で指定できます。例のように省略すると昇順です。降順にしたければ「ORDER BY age DESC」と書きます。SQL 文以外は先の例と同じです。

php age が 20 以上 30 未満を age でソートして並べる

«sample» select/between_sort.php

```
35: $sql = "SELECT * FROM member WHERE age >= 20 AND age < 30 ORDER BY age";
```

年齢が 20 ~ 29 年齢順 (昇順)

レコードを取り出す (BETW × + − □ ×)

ect/between_sort.php ☆ ...

データベースtestdbに接続しました。

ID	名前	年齢	性別
4	高峯信夫	23	男
2	塩田香織	26	女
8	井上珠理	27	女
7	佐々木伸吾	28	男

20 代を選び出し、年齢順で表示します

年齢が 20 から 29 の間という式は、BETWEEN a AND b を使って次のように書くこともできます。

php age が 20 以上 30 未満を age でソートして並べる

«sample» select/between_sort2.php

```
35: $sql = "SELECT * FROM member WHERE age BETWEEN 20 AND 29 ORDER BY age";
```

名前の部分一致検索を行う

部分一致検索では % と LIKE を利用します。次の SQL 文では名前に「木」の文字が含まれている人を選び出します。次の式の「LIKE '%木 %」の % は部分一致検索のための記号です。「'木 %」ならば「木村」のように「木」からはじまる名前を検索します。ここでは「'%木 %」のように「木」の前後に % が付いているので、「木」が含まれている名前を検索します。

php 名前に「木」の文字が含まれている人を選び出す

«sample» select/like.php

```
35: $sql = "SELECT * FROM member WHERE name LIKE '%木%'";
```

レコードを取り出す (LIKE) × + − □ ×

← → ↺ | 3/13-3/select/like.php | ☆ | ...

データベースtestdbに接続しました。

ID	名前	年齢	性別
3	雨木さくら	38	女
6	青木由香里	32	女
7	佐々木伸吾	28	男

名前に「木」が含まれる人を選び出します

データを更新する

すなわちカラムの値を更新するには UPDATE 命令を使います。書式は次のようになります。

書式 UPDATE 命令

UPDATE テーブル **SET** カラム = 値 **WHERE** 条件

名前を変更する

次の例では id が 5 の人の名前 (name カラム) を「新倉建雄」から「新倉立男」に変更します。変更後の値を確認するには、改めて SELECT 命令でレコードを取り出す SQL 文を作って実行しなければなりません。ここでは追加した id 5 の人のレコードだけを取り出して確認しています。

php name カラムの値を変更し、変更後の値を確認する

«sample» update/update_name.php

```
34: // SQL 文を作る (名前を変更する)
35: $sql = "UPDATE member SET name = '新倉立男' WHERE id = 5"; —— id 5 の人の名前を変更します
36: // プリペアドステートメントを作る
37: $stm = $pdo->prepare($sql);
38: // SQL 文を実行する
39: $stm->execute();
40:
41: // 更新後の値の確認
42: $sql = "SELECT * FROM member WHERE id = 5"; —— 変更後の id 5 の人を確認します
43: $stm = $pdo->prepare($sql);
44: $stm->execute();
45: // 結果の取得 (連想配列で受け取る)
46: $result = $stm->fetchAll(PDO::FETCH_ASSOC);
```


名前を変更する(UPDATE) × + − □ ×

← → ↻ | ate/update_name.php | ☆ | ...

データベースtestdbに接続しました。

ID	名前	年齢	性別
5	新倉立男	51	男

id 5 の人の名前を変更しました

全員の年齢に 1 を加算する

UPDATE 命令では SET する値を計算式で指定することができます。たとえば、「SET age = age + 1」のように式を書けば、全員の現在の年齢に 1 が加算されます。追加後に全員のレコードを表示して確認します。

php 全員の年齢に 1 を加算する

«sample» update/update_age

```

34: // SQL 文を作る (全員の年齢に 1 を加算する)
35: $sql = "UPDATE member SET age = age + 1"; // 全員の年齢を更新します
36: // プリペアドステートメントを作る
37: $stm = $pdo->prepare($sql);
38: // SQL 文を実行する
39: $stm->execute();
40:
41: // 更新後の確認
42: $sql = "SELECT * FROM member"; // 全員のデータを確認します
43: $stm = $pdo->prepare($sql);
44: $stm->execute();
45: // 結果の取得 (連想配列で受け取る)
46: $result = $stm->fetchAll(PDO::FETCH_ASSOC);

```

レコードを取り出す (すべて) × + − □

← → ↻ | ap13/13-2/sql/all.php | ☆ |

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女
4	高峯信夫	23	男
5	新倉立男	51	男
6	青木由香里	32	女
7	佐々木伸吾	28	男
8	井上珠理	27	女

全員の年齢を更新する(UPDATE) × + − □ ×

← → ↻ | date/update_age.php | ☆ | ..

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	33	男
2	塩田香織	27	女
3	雨木さくら	39	女
4	高峯信夫	24	男
5	新倉立男	52	男
6	青木由香里	33	女
7	佐々木伸吾	29	男
8	井上珠理	28	女

全員の年齢に 1 が加算されています

レコードを追加する

新規レコードを追加するには、INSERT 命令を使います。書式は次のようになります。カラムの値は、VALUES で対応する値を順に指定します。レコードごとに値をカッコでくくり、カンマで区切って並べれば複数のレコードを追加することができます。

書式 INSERT 命令

INSERT テーブル (カラム名 , カラム名 , ...) **VALUES** (値 , 値 , ...) , (値 , 値 , ...) , ...

次の例では 3 人のレコードを追加しています。id の値はインクリメントされるようにテーブル定義がしてあるので値を設定していません。

php メンバーを 3 人追加する

«sample» insert/insert_record.php

```

34: // SQL 文を作る (新規レコードを追加する)
35: $sql = "INSERT member (name, age, sex) VALUES
36:       ('菅田光子', 31, '女'),
37:       ('高田久美子', 44, '女'),
38:       ('青柳次郎', 35, '男');
39: // プリペアドステートメントを作る
40: $stm = $pdo->prepare($sql);
41: // SQL 文を実行する
42: $stm->execute();

```

追加するレコードの各カラムの値を並べます

レコードを追加する(INSERT × + − □ ×

← → ↻ | localhost:8888/php_nc | ☆ | ...

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	33	男
2	塩田香織	27	女
3	雨木さくら	39	女
4	高峯信夫	24	男
5	新倉立男	52	男
6	青木由香里	33	女
7	佐々木伸吾	29	男
8	井上珠理	28	女
9	菅田光子	31	女
10	高田久美子	44	女
11	青柳次郎	35	男

3 人が追加されます

レコードを削除する

レコードの削除は DELETE 命令で行います。DELETE 命令では削除するレコードの条件を WHERE で指定しないとテーブルの全レコードを削除してしまうので注意が必要です。次の例では男性のレコードを全員削除します。削除を行う前にバックアップを行っておくと安心です。レコードデータのバックアップは、エクスポートタブで行えます。書き出しておけば、インポートタブで読み込み直せます (P.523)。あるいは、操作タブにある機能を使ってテーブルごと複製しておくこともできます (次ページ)。

書式 DELETE 命令

DELETE FROM テーブル **WHERE** 条件

php 男性のレコードを削除する

«sample» delete/delete_record.php

```
34: // SQL 文を作る (男性を削除)
35: $sql = "DELETE FROM member WHERE sex = '男'";
36: // プリペアドステートメントを作る
37: $stm = $pdo->prepare($sql);
38: // SQL 文を実行する
39: $stm->execute();
```

レコードを削除する(DELETE × + - □ ×

← → ↻ | te/delete_record.php 📖 ☆ | ...

データベースtestdbに接続しました。

ID	名前	年齢	性別
2	塩田香織	27	女
3	雨木さくら	39	女
6	青木由香里	33	女
8	井上珠理	28	女
9	菅田光子	31	女
10	高田久美子	44	女

男性はすべて削除されました

NOTE

テーブルを複製する

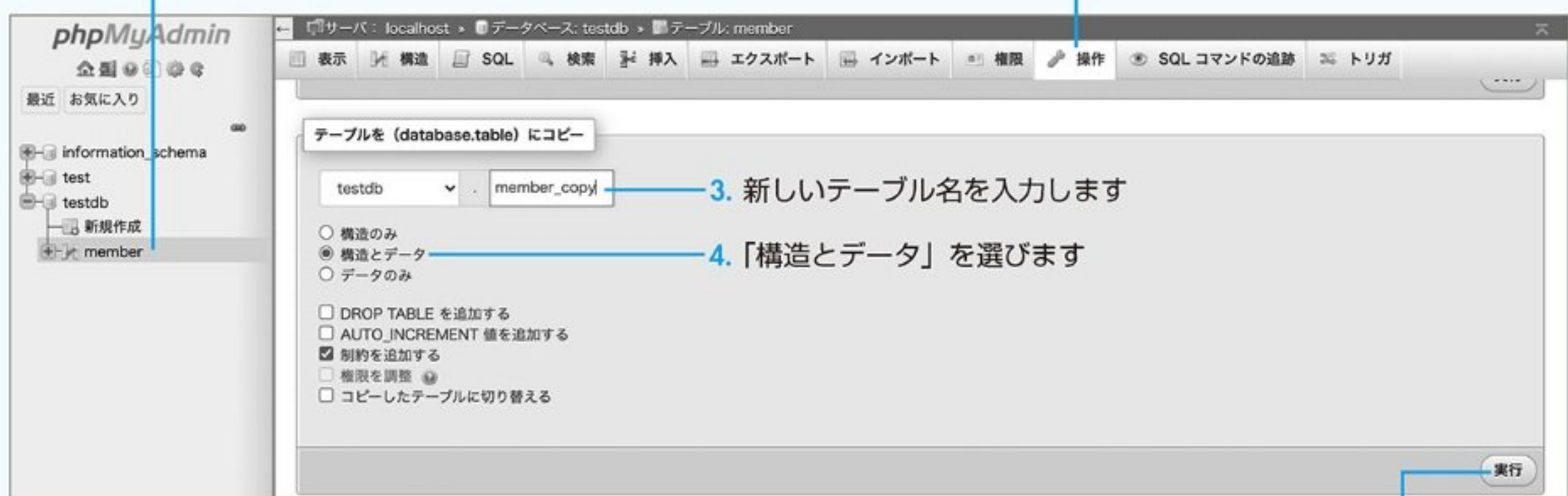
操作タブにはテーブルを別のデータベースに移動したり、複製したりする機能があります。複製の機能を利用して、member テーブルのデータをバックアップしてみましょう。

ナビゲーションパネルで member テーブルを選択し、操作タブを開きます。下にスクロールして「テーブルを (database.table) にコピー」を表示し、testdb データベースの member_copy テーブルに複製されるように複製先を指定しています。テーブルの構造のみ、構造とデータ、データのみを複製することを選べるので、ここでは構造とデータの両方を複製する選択にします。「実行」ボタンをクリックすると member テーブルを複製した member_copy テーブルがナビゲーションパネルに追加されます。member_copy テーブルにはデータも複製されています。

なお、不要になったテーブルの削除はデータベースの構造タブで行うことができ、テーブル名の変更は該当テーブルの操作タブの「テーブルオプション」で行えます。

1. member テーブルを選択します

2. 操作タブを開きます



3. 新しいテーブル名を入力します

4. 「構造とデータ」を選びます

5. クリックして実行します



6. 複製されたテーブルが追加されます

テーブルの構造とデータが複製されています

Section 13-4

フォーム入力から MySQL を利用する

フォーム入力を使って MySQL データベースを検索したり、レコードを追加したりすることがよくあります。このような場合にプリペアドステートメントのバインド機能を使うことで、効率的な SQL 文の作成と SQL エスケープによるセキュリティ対策を兼ねることができます。

SQL 文でプレースホルダを使う

前節でもデータベースに対して直接 SQL 文を実行するのではなく、プリペアドステートメントを実行する方法で MySQL を操作してきました。これまでは SQL の命令文の WHERE の条件値をそのまま書いてきましたが、ここにプレースホルダを使うことでプリペアドステートメントを使い回したり、フォームから入力された値を代入するといった利便性が出てきます。プレースホルダは SQL インジェクション対策としても有効です（P.563）。

レコードを取り出す (すべて) × +

← → ↺ | ap13/13-2/sql/all.php ☆

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女
4	高峯信夫	23	男
5	新倉建雄	43	男
6	青木由香里	32	女
7	佐々木伸吾	28	男
8	井上珠理	27	女
9	桜井信治	19	男
10	菅田光子	31	女
11	高田久美子	44	女

→

レコードを取り出す (プレースホルダ) × +

← → ↺ | fValue/bindValue.php ☆

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	32	男
7	佐々木伸吾	28	男

25 歳以上 40 歳以下の男性を選び出します

次のコードでは、SQL 文にプレースホルダを使い、後からその値をバインドしています。プレースホルダは変数、バインドは代入と考えるとわかりやすいでしょう。前後のコードはこれまでと変わりありません。

php 25 歳から 40 歳の男性を選び出す

«sample» bindValue/bindValue.php

```
01: <?php
02: require_once("../lib/util.php");
03: // データベースユーザ
04: $user = 'testuser';
05: $password = 'pw4testuser';
06: // 利用するデータベース
07: $dbName = 'testdb';
```


Section 13-4

フォーム入力から MySQL を利用する

フォーム入力を使って MySQL データベースを検索したり、レコードを追加したりすることがよくあります。このような場合にプリペアドステートメントのバインド機能を使うことで、効率的な SQL 文の作成と SQL エスケープによるセキュリティ対策を兼ねることができます。

SQL 文でプレースホルダを使う

前節でもデータベースに対して直接 SQL 文を実行するのではなく、プリペアドステートメントを実行する方法で MySQL を操作してきました。これまでは SQL の命令文の WHERE の条件値をそのまま書いてきましたが、ここにプレースホルダを使うことでプリペアドステートメントを使い回したり、フォームから入力された値を代入するといった利便性が出てきます。プレースホルダは SQL インジェクション対策としても有効です（P.563）。

レコードを取り出す (すべて) × +

← → ↺ | ap13/13-2/sql/all.php ☆

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女
4	高峯信夫	23	男
5	新倉建雄	43	男
6	青木由香里	32	女
7	佐々木伸吾	28	男
8	井上珠理	27	女
9	桜井信治	19	男
10	菅田光子	31	女
11	高田久美子	44	女

→

レコードを取り出す (プレースホルダ) × +

← → ↺ | ?Value/bindValue.php ☆

データベースtestdbに接続しました。

ID	名前	年齢	性別
1	佐藤一郎	32	男
7	佐々木伸吾	28	男

25 歳以上 40 歳以下の男性を選び出します

次のコードでは、SQL 文にプレースホルダを使い、後からその値をバインドしています。プレースホルダは変数、バインドは代入と考えるとわかりやすいでしょう。前後のコードはこれまでと変わりありません。

php 25 歳から 40 歳の男性を選び出す

«sample» bindValue/bindValue.php

```
01: <?php
02: require_once("../lib/util.php");
03: // データベースユーザ
04: $user = 'testuser';
05: $password = 'pw4testuser';
06: // 利用するデータベース
07: $dbName = 'testdb';
```



```

08: // MySQL サーバ
09: $host = 'localhost:3306';
10: // MySQL の DSN 文字列
11: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
12: ?>
13:
14: <!DOCTYPE html>
15: <html lang="ja">
16: <head>
17: <meta charset="utf-8">
18: <title>レコードを取り出す（プレースホルダを使う）</title>
19: <link href="../../../css/style.css" rel="stylesheet">
20: <!-- テーブル用のスタイルシート -->
21: <link href="../../../css/tablestyle.css" rel="stylesheet">
22: </head>
23: <body>
24: <div>
25:     <?php
26:     //MySQL データベースに接続する
27:     try {
28:         $pdo = new PDO($dsn, $user, $password);
29:         // プリペアドステートメントのエミュレーションを無効にする
30:         $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
31:         // 例外がスローされる設定にする
32:         $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
33:         echo " データベース {$dbName} に接続しました。", "<br>";
34:         // SQL 文を作る（プレースホルダを使った式）
35:         $sql = "SELECT * FROM member
36:         WHERE age >= :min AND age <= :max AND sex = :sex";
37:         // プリペアドステートメントを作る
38:         $stm = $pdo->prepare($sql);
39:         // プレースホルダに値をバインドする
40:         $stm->bindValue(':min', 25, PDO::PARAM_INT);
41:         $stm->bindValue(':max', 40, PDO::PARAM_INT);
42:         $stm->bindValue(':sex', '男', PDO::PARAM_STR);
43:         // SQL 文を実行する
44:         $stm->execute();
45:         // 結果の取得（連想配列で受け取る）
46:         $result = $stm->fetchAll(PDO::FETCH_ASSOC);
47:         // テーブルのタイトル行
48:         echo "<table>";
49:         echo "<thead><tr>";
50:         echo "<th>", "ID", "</th>";
51:         echo "<th>", "名前", "</th>";
52:         echo "<th>", "年齢", "</th>";
53:         echo "<th>", "性別", "</th>";
54:         echo "</tr></thead>";
55:         // 値を取り出して行に表示する
56:         echo "<tbody>";
57:         foreach ($result as $row){
58:             // 1行ずつテーブルに入れる
59:             echo "<tr>";
60:             echo "<td>", es($row['id']), "</td>";
61:             echo "<td>", es($row['name']), "</td>";
62:             echo "<td>", es($row['age']), "</td>";
63:             echo "<td>", es($row['sex']), "</td>";
64:             echo "</tr>";
65:         }
66:         echo "</tbody>";
67:         echo "</table>";

```

——— プレースホルダを使った SQL 文を作ります

——— プレースホルダに値をバインド（代入）します

——— SQL 文を実行します


```

68:     } catch (Exception $e) {
69:         echo '<span class="error"> エラーがありました。</span><br>';
70:         echo $e->getMessage();
71:         exit();
72:     }
73:     ?>
74: </div>
75: </body>
76: </html>

```

年齢と性別にプレースホルダを使う

この例の SQL 文では、age カラムの下限を `:min`、上限を `:max`、sex カラムの値を `:sex` というプレースホルダをそれぞれ使って指定する文になっています。プレースホルダの値が決まっていない状態のまま、`prepare()` で SQL 文をプリペアドステートメントに変換します。

php 年齢の範囲と性別にプレースホルダを使った SQL 文

«sample» bindValue/bindValue.php

```

34:     // SQL 文を作る (プレースホルダを使った式)
35:     $sql = "SELECT * FROM member
36:     WHERE age >= :min AND age <= :max AND sex = :sex";
37:     // プリペアドステートメントを作る
38:     $stm = $pdo->prepare($sql);

```

3つのプレースホルダを使っています

プレースホルダに値をバインドする

プリペアドステートメントを作った後から、プレースホルダに値をバインドします。変数に値を代入するのと同じ考え方です。バインドは `bindValue()` で行います。`bindValue()` の書式は次のとおりです。

書式 プレースホルダに値をバインドする

bindValue(プレースホルダ, 値, 値の型)

値のデータ型は PDO クラスのクラス定数で指定します。よく利用するのは次の定数です。

定数	PHP でのデータ型	MySQL などでのデータ型
PDO::PARAM_STR	string	VARCHAR、TEXT などの文字列型
PDO::PARAM_INT	int、float などの数値	INT、FLOAT などの数値型
PDO::PARAM_BOOL	boolean (論理値)	論理値
PDO::PARAM_LOB	string	BLOB などのラージオブジェクト型
PDO::PARAM_NULL	null	NULL

この例では年齢の範囲を指定するために `:min`、`:max`、性別を指定するために `:sex` のプレースホルダを使っています。`:min` には 25、`:max` には 40、`:sex` には「男」をそれぞれバインド (代入) します。

php プレースホルダに実際の値をバインドする

«sample» bindValue/bindValue.php

```

39: // プレースホルダに値をバインドする
40: $stm->bindValue(':min', 25, PDO::PARAM_INT);
41: $stm->bindValue(':max', 40, PDO::PARAM_INT);
42: $stm->bindValue(':sex', '男', PDO::PARAM_STR);

```

```
$sql = "SELECT * FROM member WHERE age >= :min AND age <= :max AND sex = :sex";
```

25

40

男

各プレースホルダに値をバインドします

これらの値をバインドすると、プレースホルダを使った SQL 文は、次の SQL 文と同じになります。つまり、25 歳から 40 歳までの男性を選び出します。

```
$sql = "SELECT * FROM member WHERE age >= 25 AND age <= 40 AND sex = '男'";
```

データベースをフォームから検索する

次の例ではフォームから入力された値でデータベースを検索します。フォームから入力された値を SQL 文にバインドする処理になります。

名前を検索するフォームを作る

この例では名前を部分一致検索します。検索する名前を入力するフォームを作り、「検索する」ボタンがクリックされたら、入力された値を search.php に POST します。例では名前に「田」の文字が含まれる人を探し出します。

名前に「田」の文字が含まれる人を探します

クリックすると入力した文字が search.php に POST されます

名前に「田」が含まれている人が選び出されます

ID	名前	年齢	性別
2	塩田香織	26	女
10	菅田光子	31	女
11	高田久美子	44	女

検索フォームに戻ります

html 名前を検索するフォームを作る

«sample» form/searchform.html

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title> 名前検索 </title>
06: <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:   <!-- 入力フォームを作る -->
11:   <form method="POST" action="search.php">
12:     <ul>
13:       <li>
14:         <label> 名前を検索します (部分一致) : <br>
15:         <input type="text" name="name" placeholder=" 名前を入れてください。 ">
16:         </label>
17:       </li>
18:       <li><input type="submit" value=" 検索する "></li>
19:     </ul>
20:   </form>
21: </div>
22: </body>
23: </html>

```

POST された値でデータベースを検索する

フォームから値が POST されたならば \$_POST 変数から値を取り出し(📖 P.300)、SQL 文の :name プレースホルダに値をバインドして名前の検索を行います。

php POST された値でデータベースを検索する

«sample» form/search.php

```

01: <?php
02: require_once("../lib/util.php");
03: $gobackURL = "searchform.html";
04:
05: // 文字エンコードの検証
06: if (!cken($_POST)){
07:   header("Location:{$gobackURL}");
08:   exit();
09: }
10:
11: // name が未設定、空のときはエラー
12: if (empty($_POST)){
13:   header("Location:searchform.html");
14:   exit();
15: } else if (!isset($_POST["name"]) || ($_POST["name"] === "")){
16:   header("Location:{$gobackURL}");
17:   exit();
18: }
19:
20: // データベースユーザ
21: $user = 'testuser';
22: $password = 'pw4testuser';

```

POST された値のチェック


```

23: // 利用するデータベース
24: $dbName = 'testdb';
25: // MySQL サーバ
26: $host = 'localhost:3306';
27: // MySQL の DSN 文字列
28: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
29: ?>
30:
31: <!DOCTYPE html>
32: <html lang="ja">
33: <head>
34: <meta charset="utf-8">
35: <title> 名前検索 </title>
36: <link href="../../../css/style.css" rel="stylesheet">
37: <!-- テーブル用のスタイルシート -->
38: <link href="../../../css/tablestyle.css" rel="stylesheet">
39: </head>
40: <body>
41: <div>
42:     <?php
43:     $name = $_POST["name"]; // POST された名前を取り出します
44:     //MySQL データベースに接続する
45:     try {
46:         $pdo = new PDO($dsn, $user, $password); // データベースへの接続
47:         // プリペアドステートメントのエミュレーションを無効にする
48:         $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
49:         // 例外がスローされる設定にする
50:         $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
51:         // SQL 文を作る
52:         $sql = "SELECT * FROM member WHERE name LIKE (:name)"; // SQL 文の作成
53:         // プリペアドステートメントを作る // プレースホルダ
54:         $stm = $pdo->prepare($sql); // プリペアドステートメントを作る
55:         // プレースホルダに値をバインドする
56:         $stm->bindValue(':name', "%{$name}%", PDO::PARAM_STR);
57:         // SQL 文を実行する プレースホルダ // POST された名前をバインドします
58:         $stm->execute(); // SQL 文の実行
59:         // 結果の取得（連想配列で受け取る）
60:         $result = $stm->fetchAll(PDO::FETCH_ASSOC);
61:         if(count($result)>0){
62:             echo " 名前に「{$name}」が含まれているレコード ";
63:             // テーブルのタイトル行
64:             echo "<table>";
65:             echo "<thead><tr>";
66:             echo "<th>", "ID", "</th>";
67:             echo "<th>", "名前", "</th>";
68:             echo "<th>", "年齢", "</th>";
69:             echo "<th>", "性別", "</th>";
70:             echo "</tr></thead>";
71:             // 値を取り出して行に表示する
72:             echo "<tbody>";
73:             foreach ($result as $row){
74:                 // 1行ずつテーブルに入れる
75:                 echo "<tr>";
76:                 echo "<td>", es($row['id']), "</td>";
77:                 echo "<td>", es($row['name']), "</td>";
78:                 echo "<td>", es($row['age']), "</td>";
79:                 echo "<td>", es($row['sex']), "</td>";
80:                 echo "</tr>";
81:             }
82:             echo "</tbody>";

```

検索結果を表示します


```

83:         echo "</table>";
84:     } else {
85:         echo " 名前に「{$name}」は見つかりませんでした。";
86:     }
87: } catch (Exception $e) {
88:     echo '<span class="error">エラーがありました。</span><br>';
89:     echo $e->getMessage();
90: }
91: ?>
92: <hr>
93: <p><a href="<?php echo $gobackURL ?>">戻る </a></p>
94: </div>
95: </body>
96: </html>

```

名前の部分一致検索

名前の部分一致検索は LIKE 句を使って行います。検索する文字は :name プレースホルダにした SQL 文でプリペアドステートメントを作ります。

bindValue() では、部分一致検索にするために :name に "%{\$name}%" をバインドします。変数 \$name には、POST された名前を代入しておきます。(部分一致検索 P.551)

php POST された名前を :name プレースホルダにバインドして検索する

«sample» form/search.php

```

51: // SQL 文を作る
52: $sql = "SELECT * FROM member WHERE name LIKE(:name)";
53: // プリペアドステートメントを作る
54: $stm = $pdo->prepare($sql);
55: // プレースホルダに値をバインドする
56: $stm->bindValue(':name', "%{$name}%", PDO::PARAM_STR);
57: // SQL 文を実行する
58: $stm->execute();

```



セキュリティ対策

SQL インジェクション対策

フォーム入力などから悪意のある SQL 文を送信し、データベースをハッキングする行為は SQL インジェクションと呼ばれます。SQL インジェクションに対抗するには、この節で説明したようにプレースホルダを使って SQL 文を記述してプリペアドステートメントを作ります。プレースホルダの値を bindValue() を使ってバインドすることで、SQL エスケープも同時に行われます。この手順は SQL インジェクション対策として有効な手段になります。

新規レコードの入力フォームを作る

次の例ではデータベース testdb の「名前、年齢、性別」の値をフォーム入力してレコードを追加します。id はオートインクリメントの設定にしてあるので入力しません。入力フォームを作るコードは次のとおりです。

testdb に追加するレコードの値を入力します

html 新規レコードの入力フォームを作る

«sample» form/insertform.html

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>レコード追加</title>
06: <link href="../../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:   <!-- 入力フォームを作る -->
11:   <form method="POST" action="insert_member.php">
12:     <ul>
13:       <li>
14:         <label>名前:
15:         <input type="text" name="name" placeholder="名前">
16:       </li>
17:       <li>
18:         <label>年齢:
19:         <input type="number" name="age" placeholder="半角数字">
20:       </li>
21:       <li>性別:
22:         <label><input type="radio" name="sex" value="男" checked>男性</label>
23:         <label><input type="radio" name="sex" value="女">女性</label>
24:       </li>
25:       <li><input type="submit" value="追加する"></li>
26:     </ul>
27:   </form>
28: </div>
29: </body>
30: </html>

```

名前、年齢、性別を POST します

ID	名前	年齢	性別
1	佐藤一郎	32	男
2	塩田香織	26	女
3	雨木さくら	38	女
4	高峯信夫	23	男
5	新倉建雄	43	男
6	青木由香里	32	女
7	佐々木伸吾	28	男
8	井上珠理	27	女
9	桜井信治	19	男
10	菅田光子	31	女
11	高田久美子	44	女
12	朝霧晴夫	18	男

フォーム入力したレコードが追加されます

POST された値で新規レコードを追加する

フォームで入力された値は POST で渡されるので、`$_POST` 変数の値をチェックした後にデータベースに追加します。前節で説明したように、レコードを追加する SQL は `INSERT` 命令です。セルの値は「`:name, :age, :sex`」のようにプレースホルダを利用してプリペアドステートメントを作り、プレースホルダの値にフォーム入力で得られた値（`$name`、`$age`、`$sex`）をバインドして実行します。

INSERT 命令が成功したならば結果を表示する

`INSERT` 命令が返す値は実行結果が成功（`true`）か失敗（`false`）を示す論理値なので、追加後のレコードを表示するには改めて `SELECT` 命令でレコードを取り出す SQL 文を作って実行する必要があります。

php POST された値で新規レコードを追加する

«sample» form/insert_member.php

```

01: <?php
02: require_once("../lib/util.php");
03: $gobackURL = "insertform.html";
04:
05: // 文字エンコードの検証
06: if (!cken($_POST)){
07:     header("Location:{$gobackURL}");
08:     exit();
09: }
10:
11: // 簡単なエラー処理
12: $errors = [];

```



```

13: if (!isset($_POST["name"]) || ($_POST["name"] === "")) {
14:     $errors[] = "名前が空です。";
15: }
16: if (!isset($_POST["age"]) || (!ctype_digit($_POST["age"]))) {
17:     $errors[] = "年齢には数値を入れてください。";
18: }
19: if (!isset($_POST["sex"]) || !in_array($_POST["sex"], ["男", "女"])) {
20:     $errors[] = "性別が男または女ではありません。";
21: }
22:
23: // エラーがあったとき
24: if (count($errors) > 0) {
25:     echo '<ol class="error">';
26:     foreach ($errors as $value) {
27:         echo "<li>", $value, "</li>";
28:     }
29:     echo "</ol>";
30:     echo "<hr>";
31:     echo "<a href=", $gobackURL, "> 戻る </a>";
32:     exit();
33: }
34:
35: // データベースユーザ
36: $user = 'testuser';
37: $password = 'pw4testuser';
38: // 利用するデータベース
39: $dbName = 'testdb';
40: // MySQL サーバ
41: $host = 'localhost:3306';
42: // MySQL の DSN 文字列
43: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
44: ?>
45:
46: <!DOCTYPE html>
47: <html lang="ja">
48: <head>
49: <meta charset="utf-8">
50: <title>レコード追加</title>
51: <link href="../../../css/style.css" rel="stylesheet">
52: <!-- テーブル用のスタイルシート -->
53: <link href="../../../css/tablestyle.css" rel="stylesheet">
54: </head>
55: <body>
56: <div>
57:     <?php
58:     $name = $_POST["name"];
59:     $age = $_POST["age"];
60:     $sex = $_POST["sex"];
61:     //MySQL データベースに接続する
62:     try {
63:         $pdo = new PDO($dsn, $user, $password);
64:         // プリペアドステートメントのエミュレーションを無効にする
65:         $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
66:         // 例外がスローされる設定にする
67:         $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
68:
69:         // SQL 文を作る
70:         $sql = "INSERT INTO member (name, age, sex) VALUES (:name, :age, :sex)";
71:         // プリペアドステートメントを作る
72:         $stm = $pdo->prepare($sql);

```

—— フォームから POST された値を取り出します

—— レコードを追加します

—— セルに代入するブレースホルダ


```

73: // プレースホルダに値をバインドする
74: $stm->bindValue(':name', $name, PDO::PARAM_STR);
75: $stm->bindValue(':age', $age, PDO::PARAM_INT);
76: $stm->bindValue(':sex', $sex, PDO::PARAM_STR);
77: // SQL 文を実行する
78: if ($stm->execute()){
79:     // レコード追加後のレコードリストを取得する
80:     $sql = "SELECT * FROM member";
81:     // プリペアドステートメントを作る
82:     $stm = $pdo->prepare($sql);
83:     // SQL 文を実行する
84:     $stm->execute();
85:     // 結果の取得 (連想配列で受け取る)
86:     $result = $stm->fetchAll(PDO::FETCH_ASSOC);
87:     // テーブルのタイトル行
88:     echo "<table>";
89:     echo "<thead><tr>";
90:     echo "<th>", "ID", "</th>";
91:     echo "<th>", " 名前 ", "</th>";
92:     echo "<th>", " 年齢 ", "</th>";
93:     echo "<th>", " 性別 ", "</th>";
94:     echo "</tr></thead>";
95:     // 値を取り出して行に表示する
96:     echo "<tbody>";
97:     foreach ($result as $row) {
98:         // 1行ずつテーブルに入れる
99:         echo "<tr>";
100:         echo "<td>", es($row['id']), "</td>";
101:         echo "<td>", es($row['name']), "</td>";
102:         echo "<td>", es($row['age']), "</td>";
103:         echo "<td>", es($row['sex']), "</td>";
104:         echo "</tr>";
105:     }
106:     echo "</tbody>";
107:     echo "</table>";
108: } else {
109:     echo '<span class="error"> 追加エラーがありました。</span><br>';
110: };
111: } catch (Exception $e) {
112:     echo '<span class="error"> エラーがありました。</span><br>';
113:     echo $e->getMessage();
114: }
115: ?>
116: <hr>
117: <p><a href="<?php echo $gobackURL ?>">戻る </a></p>
118: </div>
119: </body>
120: </html>

```

—— プレースホルダに POST された
値を代入します

—— SQL 文を実行します

—— レコードを追加する SQL 文が成功したならば、
すべてのレコードを表示します。

Section 13-5

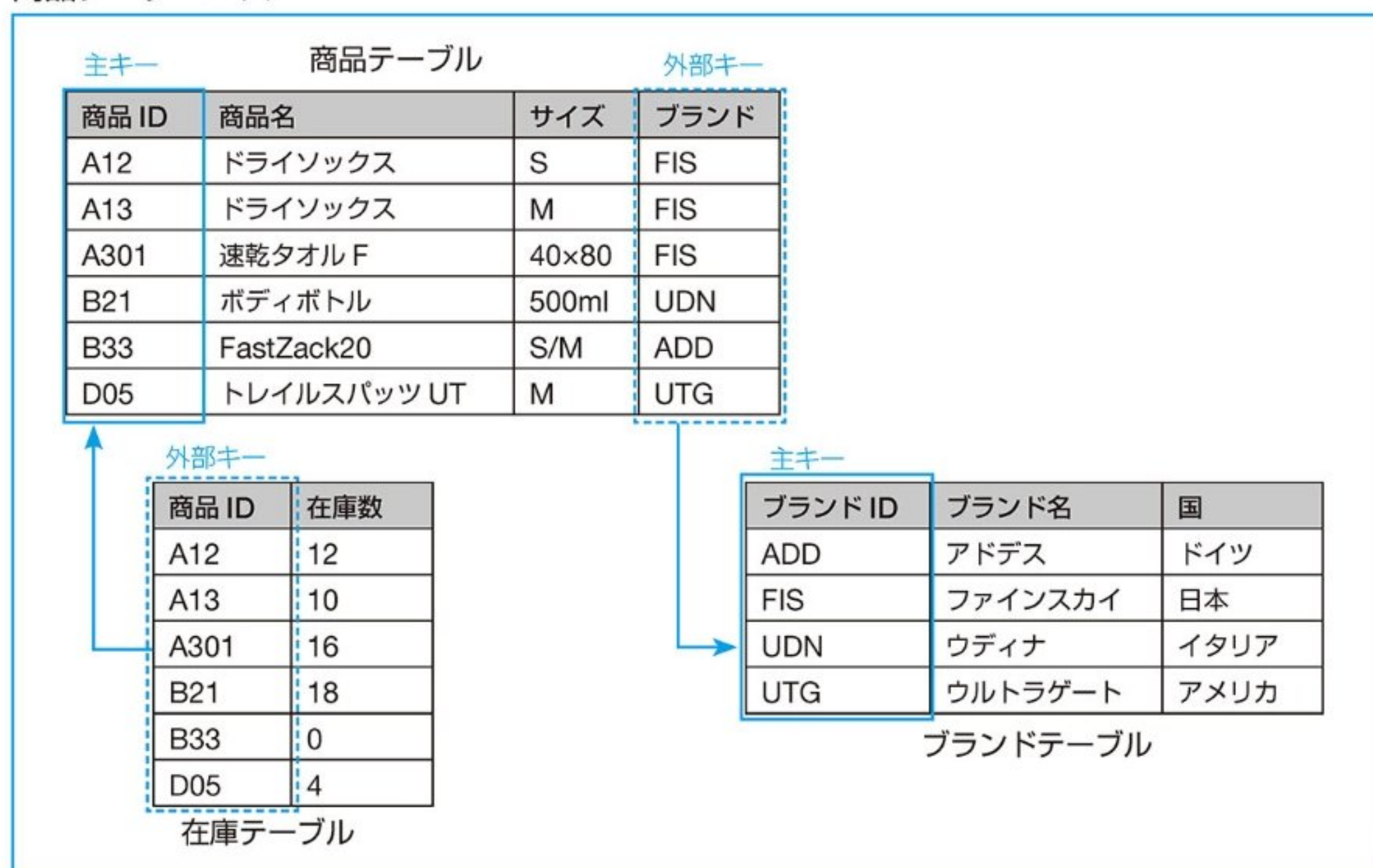
リレーショナルデータベースのレコードを取り出す

リレーショナルデータベースでは外部キーでリレーションしているテーブルの値を取り出す操作があります。また、複数のテーブルを使うとカラム名が重複している場合にそれらを区別する必要があります。この節では商品データベースを使ってこれらの解決方法を説明します。

商品データベース

この節では Section12-3 で作成した商品データベース (inventory) を使って説明します (P.510)。データベース inventory には、商品テーブル (goods)、ブランドテーブル (brand)、在庫テーブル (stock) の3つのテーブルがあります。

商品データベース



データベースユーザを追加する

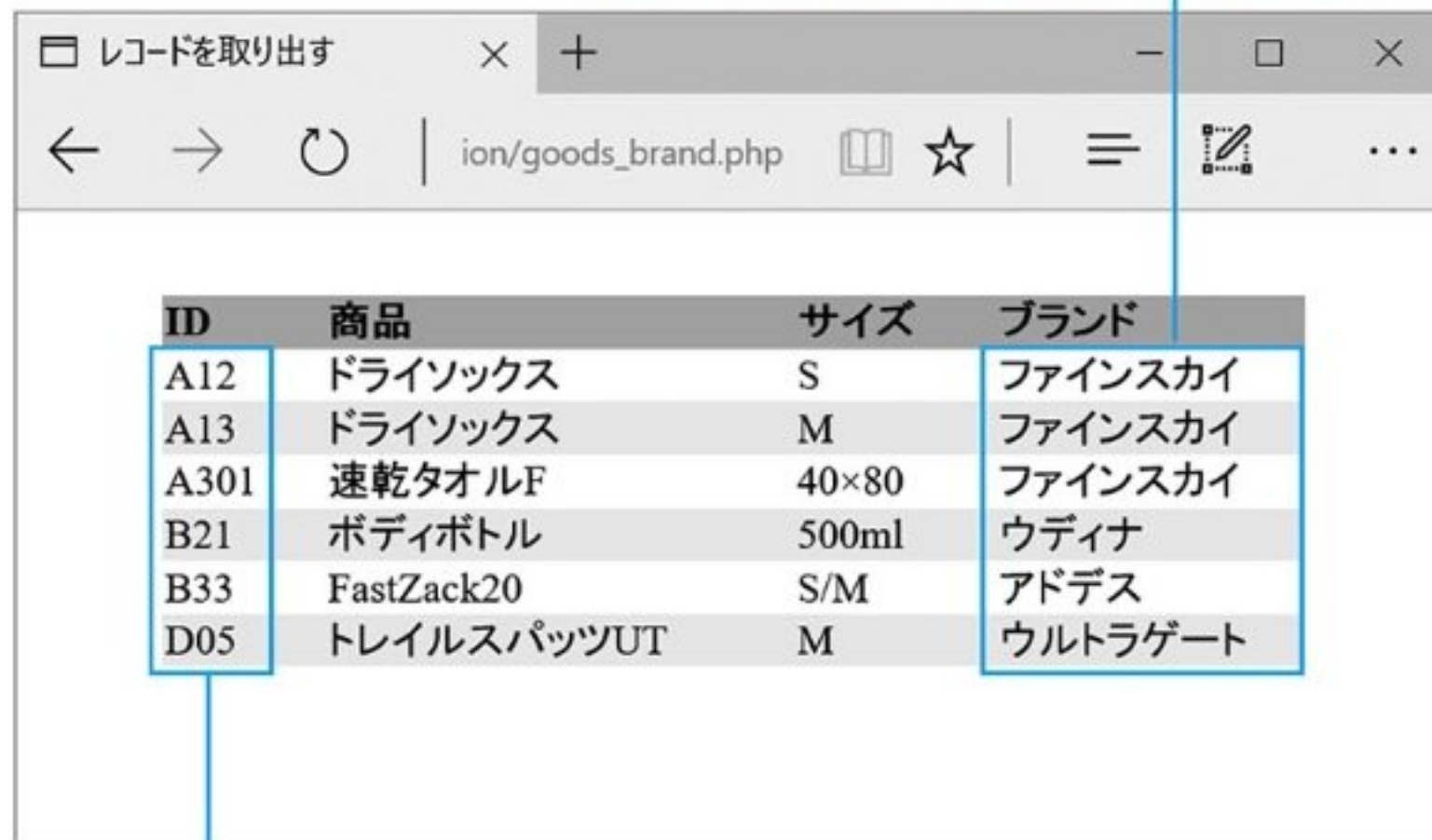
商品データベース inventory を利用するために、データベースユーザ inventoryuser を追加しておきます。データベースユーザを追加する方法は「Section13-1 データベースユーザを追加する」を参考にしてください (P.526)。

データベース：inventory
 ユーザ名：inventoryuser
 パスワード：pw4inventoryuser

商品のブランド名をブランド ID で調べて表示する

次の例では商品テーブルの内容に加えて、外部キーのブランド ID を使ってブランドテーブルからブランド名を調べて表示しています。

ブランド ID でブランド名を調べて表示します。



ID	商品	サイズ	ブランド
A12	ドライソックス	S	ファインスカイ
A13	ドライソックス	M	ファインスカイ
A301	速乾タオルF	40×80	ファインスカイ
B21	ボディボトル	500ml	ウディナ
B33	FastZack20	S/M	アドデス
D05	トレイルスパッツUT	M	ウルトラゲート

レコードは商品 ID の順に並んでいます

php 商品テーブルのブランド ID からブランド名を調べて表にする

«sample» relation/goods_brand.php

```

01: <?php
02: require_once("../../lib/util.php");
03: // データベースユーザ
04: $user = 'inventoryuser';
05: $password = 'pw4inventoryuser';
06: // 利用するデータベース
07: $dbName = 'inventory';
08: // MySQL サーバ
09: $host = 'localhost:3306';
10: // MySQL の DSN 文字列
11: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
12: ?>
13:
14: <!DOCTYPE html>
15: <html lang="ja">
16: <head>
17: <meta charset="utf-8">
18: <title>レコードを取り出す</title>
19: <link href="../css/style.css" rel="stylesheet">
20: <!-- テーブル用のスタイルシート -->
21: <link href="../../css/tablestyle2.css" rel="stylesheet">
22: </head>
23: <body>
24: <div>

```



```

25: <?php
26: //MySQL データベースに接続する
27: try {
28:     $pdo = new PDO($dsn, $user, $password);————データベースに接続します
29:     // プリペアドステートメントのエミュレーションを無効にする
30:     $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
31:     // 例外がスローされる設定にする
32:     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
33:     // SQL 文を作る
34:     $sql = "SELECT goods.id as goods_id, goods.name as goods_name,
35:         goods.size, brand.name as brand_name
36:     FROM goods, brand
37:     WHERE goods.brand = brand.id
38:     ORDER BY goods_id";
39:     // プリペアドステートメントを作る
40:     $prepare = $pdo->prepare($sql);————プリペアドステートメントを作ります
41:     // SQL 文を実行する
42:     $prepare->execute();————SQL 文を実行します
43:     // 結果の取得（連想配列で受け取る）
44:     $result = $prepare->fetchAll(PDO::FETCH_ASSOC);
45:     // テーブルのタイトル行
46:     echo "<table>";
47:     echo "<thead><tr>";
48:     echo "<th>", "ID", "</th>";
49:     echo "<th>", "商品", "</th>";
50:     echo "<th>", "サイズ", "</th>";
51:     echo "<th>", "ブランド", "</th>";
52:     echo "</tr></thead>";
53:     // 値を取り出して行に表示する
54:     echo "<tbody>";
55:     foreach ($result as $row){
56:         // 1行ずつテーブルに入れる
57:         echo "<tr>";
58:         echo "<td>", es($row['goods_id']), "</td>";
59:         echo "<td>", es($row['goods_name']), "</td>";
60:         echo "<td>", es($row['size']), "</td>";
61:         echo "<td>", es($row['brand_name']), "</td>";
62:         echo "</tr>";
63:     }
64:     echo "</tbody>";
65:     echo "</table>";
66: } catch (Exception $e) {
67:     echo '<span class="error">エラーがありました。</span><br>';
68:     echo $e->getMessage();
69:     exit();
70: }
71: ?>
72: </div>
73: </body>
74: </html>

```

2つのテーブルから値を取り出す SQL 文を作ります

商品テーブルとブランドテーブルの同名のカラム名を区別する

商品 ID、商品名、サイズ、ブランド名という並びの表を作る SQL 文を見てみましょう。少し長いですが、構文としては「SELECT カラム FROM テーブル WHERE 条件 ORDER BY 並び順」になっています。

php 商品データの表を作る SQL 文

«sample» relation/goods_brand.php

```

33:      // SQL 文を作る
34:      $sql = "SELECT goods.id as goods_id, goods.name as goods_name,
35:             goods.size, brand.name as brand_name
36:      FROM goods, brand——— 2つのテーブルから取り出す
37:      WHERE goods.brand = brand.id——— 2つのテーブルでブランドID が一致しているレコード
38:      ORDER BY goods_id";——— 商品 ID 順に並ぶ

```

まず、FROM の部分を見てください。今から作る表では商品テーブルとブランドテーブルの2つの表から値を取り出します。そこで「FROM goods, brand」と指定します。

次に取り出すレコードを WHERE で指定します。「goods.brand = brand.id」とすると、商品テーブルに入っているブランド ID と一致するブランドテーブルのレコードが取り出されます。

レコードから取り出すカラムを指定する

レコードから取り出す値は SELECT でカラムを指定します。これまではすべてのカラムの値を取り出す * を書いていましたが、ここではカラムを個別に指定します。ここで問題となるのが、商品テーブルとブランドテーブルには同じ名前のカラムが存在するということです。具体的には id カラムと name カラムがど両方のテーブルにあります。そこで goods テーブルの name カラムならば「goods.name」、brand テーブルの name カラムならば「brand.name」のように書くことで両者が混乱しないようにします。

as 演算子で名前を付け直す

さらに「goods.id as goods_id」のように as 演算子を使って、goods テーブルの id を goods_id の名前で扱えるようにしています。同様に「goods.name as goods_name」、「brand.name as brand_name」とすることで、goods テーブルの name は goods_name、brand テーブルの name は brand_name で扱えるようにしています。

値を表示する

as 演算子で付けた名前は、カラムから取り出した値を表示する際に利用します。ORDER BY で goods_id を指定しているので、商品 ID 順に並んだ表になります。

php 取り出した値を表示する

«sample» relation/goods_brand.php

```

55:      foreach ($result as $row){
56:          // 1行ずつテーブルに入れる
57:          echo "<tr>";
58:          echo "<td>", es($row['goods_id']), "</td>";
59:          echo "<td>", es($row['goods_name']), "</td>";
60:          echo "<td>", es($row['size']), "</td>";
61:          echo "<td>", es($row['brand_name']), "</td>";
62:          echo "</tr>";
63:      }

```

as 演算子で付けた名前呼び出します

3つのテーブルの値を連携する

次の例ではさらに在庫テーブルから商品の在庫数を取り出して表に追加します。この表を作るには、商品テーブル、ブランドテーブル、在庫テーブルの3つのテーブルの値を連携する必要があります。

ブランドIDでブランド名を調べて表示します。

外部キーの値を取り出す × +

ods_brand_stock.php ☆

≡

ID	商品	サイズ	ブランド	在庫
B33	FastZack20	S/M	アドデス	0
D05	トレイルスパッツUT	M	ウルトラゲート	4
A13	ドライソックス	M	ファインスカイ	10
A12	ドライソックス	S	ファインスカイ	12
B21	ボディボトル	500ml	ウディナ	18
A301	速乾タオルF	40×80	ファインスカイ	16

レコードは商品名の順に並んでいます 商品IDで在庫数を調べて表示します

先のコードと基本的には同じですが、SQL文と取り出した値を表示する部分が違ってきます。

php 商品ID、商品名、サイズ、ブランド名、在庫数を合わせた表を表示する

«sample» relation/goods_brand_stock.php

```
01: <?php
02: require_once("../lib/util.php");
03: // データベースユーザ
04: $user = 'inventoryuser';
05: $password = 'pw4inventoryuser';
06: // 利用するデータベース
07: $dbName = 'inventory';
08: // MySQL サーバ
09: $host = 'localhost:3306';
10: // MySQL の DSN 文字列
11: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
12: ?>
13:
14: <!DOCTYPE html>
15: <html lang="ja">
16: <head>
17: <meta charset="utf-8">
18: <title> 外部キーの値を取り出す </title>
19: <link href="../css/style.css" rel="stylesheet">
20: <!-- テーブル用のスタイルシート -->
21: <link href="../../css/tablestyle2.css" rel="stylesheet">
22: </head>
23: <body>
24: <div>
```



```

25: <?php
26: //MySQL データベースに接続する
27: try {
28:     $pdo = new PDO($dsn, $user, $password);
29:     // プリペアドステートメントのエミュレーションを無効にする
30:     $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
31:     // 例外がスローされる設定にする
32:     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
33:     // SQL 文を作る
34:     $sql = "SELECT goods.id as goods_id, goods.name as goods_name, goods.size,
35:           brand.name as brand_name, stock.quantity
36:           FROM goods, brand, stock
37:           WHERE goods.brand = brand.id AND goods.id = stock.goods_id
38:           ORDER BY goods_name";
39:     // プリペアドステートメントを作る
40:     $stm = $pdo->prepare($sql);
41:     // SQL 文を実行する
42:     $stm->execute();
43:     // 結果の取得（連想配列で受け取る）
44:     $result = $stm->fetchAll(PDO::FETCH_ASSOC);
45:     // テーブルのタイトル行
46:     echo "<table>";
47:     echo "<thead><tr>";
48:     echo "<th>", "ID", "</th>";
49:     echo "<th>", "商品", "</th>";
50:     echo "<th>", "サイズ", "</th>";
51:     echo "<th>", "ブランド", "</th>";
52:     echo "<th>", "在庫", "</th>";
53:     echo "</tr></thead>";
54:     // 値を取り出して行に表示する
55:     echo "<tbody>";
56:     foreach ($result as $row){
57:         // 1行ずつテーブルに入れる
58:         echo "<tr>";
59:         echo "<td>", es($row['goods_id']), "</td>";
60:         echo "<td>", es($row['goods_name']), "</td>";
61:         echo "<td>", es($row['size']), "</td>";
62:         echo "<td>", es($row['brand_name']), "</td>";
63:         echo "<td>", es($row['quantity']), "</td>";
64:         echo "</tr>";
65:     }
66:     echo "</tbody>";
67:     echo "</table>";
68: } catch (Exception $e) {
69:     echo '<span class="error">エラーがありました。</span><br>';
70:     echo $e->getMessage();
71:     exit();
72: }
73: ?>
74: </div>
75: </body>
76: </html>

```

3つのテーブルから値を取り出す SQL 文を作ります

3つのテーブルから取り出した値を並べて表示します

ブランド ID に一致するブランド名、商品 ID に一致する在庫を取り出す

それでは SQL 文を見てみましょう。商品テーブル、ブランドテーブル、在庫テーブルの3つのテーブルから値を取り出すので、SELECT 命令の FROM には「goods, brand, stock」と書きます。

そして先ほどの表と同じように商品ブランドを表示するのでレコードを取り出す条件として WHERE に「goods.brand = brand.id」を指定しますが、今回は商品の在庫数も取り出すことから、「goods.id = stock.goods_id」の指定も合わせて追加します。取り出すレコードはこの2つの条件を同時に満たしている必要があることから、2つの条件は AND で連結します。

php 商品 ID、商品名、サイズ、ブランド名、在庫数を取り出す SQL

«sample» relation/goods_brand_stock.php

```
33: // SQL 文を作る
34: $sql = "SELECT goods.id as goods_id, goods.name as goods_name, goods.size,
35:         brand.name as brand_name, stock.quantity
36: FROM goods, brand, stock
37: WHERE goods.brand = brand.id AND goods.id = stock.goods_id
38: ORDER BY goods_name";
```

3つのテーブルから値を取り出す

ブランド ID が一致

商品 ID が一致

ソートキー

選び出したレコードから取り出すカラムは、商品テーブルの「goods.id、goods.name、goods.size」、ブランドテーブルの「brand.name」、在庫テーブルの「stock.quantity」です。ORDER BY で goods_name を指定しているので商品名でソートされた表になります。

Section 13-6

トランザクション処理

商品データの追加と同時に在庫データを更新するというように、複数の関連したデータ処理を行う場合は、すべての処理が成功する必要があります。このような場合には、1つでも処理が失敗したならば元の状態に戻すという仕組みが必要です。この仕組みがトランザクション処理です。

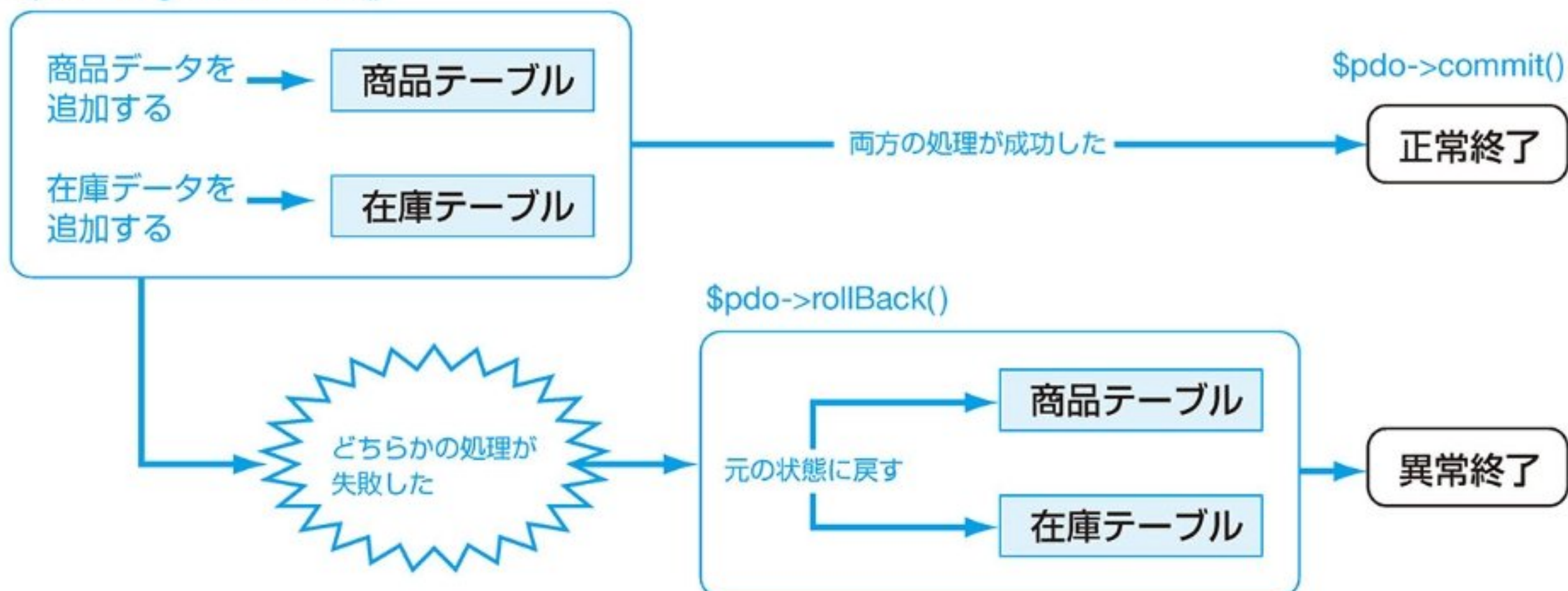
入力フォームから商品レコードを追加する

先の商品データベースを使ってトランザクション処理の例を示します。次の例では、フォーム入力を使って、商品データベースに新しい商品を追加します。このとき、商品テーブルに新規商品を追加すると同時に、在庫テーブルに在庫数を追加します。

この2つの処理はどちらかが失敗すると整合性が保てません。そこで、例外処理とトランザクション機能を合わせて、どちらかのレコード追加処理が失敗したならば、データベースを元の状態に戻せるようにします。

トランザクション処理

```
$pdo->beginTransaction()
```



入力フォームから商品データを追加する

まず、商品データを追加する入力フォームを作ります。基本的には商品テーブルに入力するデータですが、在庫テーブルに追加する在庫数も入力します。ブランドはプルダウンメニュー（P.369）からブランド名で選択しますが、実際に入力されるのはブランド ID です。

goods_brand_stock.php で確認します (P.572)

レコード追加

商品ID: DG7

商品名: サファリハット

サイズ: M/L

ブランド: ウルトラゲート

個数: 5

追加する

外部キーの値を取り出す

ID	商品	サイズ	ブランド	在庫
B33	FastZack20	S/M	アドデス	0
DG7	サファリハット	M/L	ウルトラゲート	5
D05	トレイルスパッツUT	M	ウルトラゲート	4
A12	ドライソックス	S	ファインスカイ	12
A13	ドライソックス	M	ファインスカイ	10
B21	ボディボトル	500ml	ウディナ	18
A301	速乾タオルF	40×80	ファインスカイ	16

レコードが追加されています

商品データを入力します

ブランド名を選択しますが、値には
ブランドIDが入ります

php 商品データの入力フォームを作る

«sample» transaction/insertform.php

```

01: <?php
02: require_once("../lib/util.php");
03: $gobackURL = "insertform.html";
04:
05: // データベースユーザ
06: $user = 'inventoryuser';
07: $password = 'pw4inventoryuser';
08: // 利用するデータベース
09: $dbName = 'inventory';
10: // MySQL サーバ
11: $host = 'localhost:3306';
12: // MySQL の DSN 文字列
13: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
14: //MySQL データベースに接続する
15: try {
16:     $pdo = new PDO($dsn, $user, $password); // データベースに接続します
17:     // プリペアドステートメントのエミュレーションを無効にする
18:     $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
19:     // 例外がスローされる設定にする
20:     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
21:
22:     // ブランドテーブルからブランドIDとブランド名を取り出す
23:     $sql = "SELECT id, name FROM brand";
24:     // プリペアドステートメントを作る
25:     $stm = $pdo->prepare($sql);
26:     // SQL 文を実行する
27:     $stm->execute();
28:     // 結果の取得 (連想配列で受け取る)
29:     $brand = $stm->fetchAll(PDO::FETCH_ASSOC);
30: } catch (Exception $e) {
31:     $err = '<span class="error">エラーがありました。</span><br>';
32:     $err .= $e->getMessage();
33:     exit($err);
34: }
35: ?>
36:
37: <!DOCTYPE html>
38: <html lang="ja">

```

登録済みのブランドIDとブランド名を
ブランドテーブルから取り出します


```

39: <head>
40: <meta charset="utf-8">
41: <title>レコード追加</title>
42: <link href="../../css/style.css" rel="stylesheet">
43: </head>
44: <body>
45: <div>
46:     <!-- 入力フォームを作る -->
47:     <form method="POST" action="insert_goods.php">
48:         <ul>
49:             <li>
50:                 <label>商品 ID :
51:                 <input type="text" name="id" placeholder="商品 ID">
52:                 </label>
53:             </li>
54:             <li>
55:                 <label>商品名 :
56:                 <input type="text" name="name" placeholder="商品名 ">
57:                 </label>
58:             </li>
59:             <li>
60:                 <label>サイズ :
61:                 <input type="text" name="size" placeholder="(未入力でも OK) ">
62:                 </label>
63:             </li>
64:             <li>ブランド :
65:                 <select name="brand">
66:                     <?php
67:                     // ブランドはブランドテーブルに登録してあるものから選ぶ
68:                     foreach ($brand as $row){
69:                         echo '<option value="' . $row["id"] . '>' . $row["name"] . "</option>";
70:                     }
71:                     ?>
72:                 </select>      ブランドを選択するプルダウンメニューを作ります
73:             </li>
74:             <li>
75:                 <label>個数 :
76:                 <input type="number" name="quantity" placeholder="半角数字 ">
77:                 </li>
78:             <li><input type="submit" value="追加する "></li>
79:         </ul>
80:     </form>
81: </div>
82: </body>
83: </html>

```

ブランドテーブルからブランド ID とブランド名を取り出す

商品テーブルに入力するブランド ID はブランドテーブルの外部キーなので、ブランドテーブルから選ばなければなりません。そこで入力フォームでは、プルダウンメニューからブランドを選択するとブランド ID が value に入力されるようにします。

これを行なうために、ブランドテーブルからブランド ID とブランド名を取り出します。次の SELECT 命令では WHERE の条件を付けていないので、ブランドテーブルのすべてのレコードの id カラムと name カラムの値が選択されます。

php ブランドテーブルからブランド ID とブランド名を取り出す

«sample» transaction/insertform.php

```

22: // ブランドテーブルからブランド ID とブランド名を取り出す
23: $sql = "SELECT id, name FROM brand"; // WHERE の条件がないのですべて取り出されます
24: // プリペアドステートメントを作る
25: $stm = $pdo->prepare($sql);
26: // SQL 文を実行する
27: $stm->execute();
28: // 結果の取得（連想配列で受け取る）
29: $brand = $stm->fetchAll(PDO::FETCH_ASSOC);

```

ブランドテーブルからブランド名のプルダウンメニューを作る

ブランドテーブルからブランド ID とブランド名を \$brand に取り出したならば、その値を使ってプルダウンメニューを作ります。value に入力する値はブランド ID ですが、プルダウンメニューではブランド名で選べるようにしています。

php ブランド名のプルダウンメニューを作る

«sample» transaction/insertform.php

```

64: <li> ブランド :
65:     <select name="brand">
66:         <?php
67:             // ブランドはブランドテーブルに登録してあるものから選ぶ
68:             foreach ($brand as $row){
69:                 echo '<option value="', $row["id"], '>', $row["name"], "</option>";
70:             }
71:         ?>
72:     </select>
73: </li>

```

実際の値はブランド ID
メニューにはブランド名で表示

ブランドテーブルに登録済みの
ブランドから選べるようにします

商品の追加に合わせて在庫も追加する

入力フォームの値は次の insert_goods.php に POST されます。insert_goods.php では、POST された値に基づいて商品テーブルと在庫テーブルに新規レコードを追加します。最初に書いたように、ここでどちらかの処理が失敗したならば元に戻すトランザクション処理を行います。

商品テーブル (goods)

←T→	▼	id	name	size	brand		
☐				A12	ドライソックス	S	FIS
☐				A13	ドライソックス	M	FIS
☐				A301	速乾タオルF	40×80	FIS
☐				B21	ボディボトル	500ml	UDN
☐				B33	FastZack20	S/M	ADD
☐				D05	トレイルスパッツUT	M	UTG
☐				DG7	サファリハット	M/L	UTG

POST された値でレコードを追加します

在庫テーブル (stock)

←T→				▼	goods_id	quantity	
☐		編集		コピー		削除 A12	12
☐		編集		コピー		削除 A13	10
☐		編集		コピー		削除 A301	16
☐		編集		コピー		削除 B21	18
☐		編集		コピー		削除 B33	0
☐		編集		コピー		削除 D05	4
☐		編集		コピー		削除 DG7	5

POST された値でレコードを追加します

php 商品の追加に合わせて在庫も追加する

«sample» transaction/insert_goods.php

```

01: <?php
02: require_once("../lib/util.php");
03: $gobackURL = "insertform.php";
04:
05: // 文字エンコードの検証
06: if (!cken($_POST)){
07:     header("Location:{$gobackURL}");
08:     exit();
09: }
10: ?>
11:
12: <!DOCTYPE html>
13: <html lang="ja">
14: <head>
15: <meta charset="utf-8">
16: <title>レコード追加</title>
17: <link href="../../css/style.css" rel="stylesheet">
18: </head>
19: <body>
20: <div>
21:     <?php
22:     // 簡単なエラー処理
23:     $errors = [];
24:     if (!isset($_POST["id"]) || ($_POST["id"]=="")){
25:         $errors[] = "商品 ID が空です。";
26:     }
27:     if (!isset($_POST["name"]) || ($_POST["name"]=="")){
28:         $errors[] = "商品名が空です。";
29:     }
30:     if (!isset($_POST["brand"]) || ($_POST["brand"]=="")) {
31:         $errors[] = "ブランドが空です。";
32:     }

```



```

33: if (!isset($_POST["quantity"]) || (!ctype_digit($_POST["quantity"]))) {
34:     $errors[] = "個数が整数値ではありません。";
35: }
36: // エラーがあったとき
37: if (count($errors) > 0) {
38:     echo '<ol class="error">';
39:     foreach ($errors as $value) {
40:         echo "<li>", $value, "</li>";
41:     }
42:     echo "</ol>";
43:     echo "<hr>";
44:     echo "<a href=", $gobackURL, ">戻る</a>";
45:     exit();
46: }
47:
48: // データベースユーザ
49: $user = 'inventoryuser';
50: $password = 'pw4inventoryuser';
51: // 利用するデータベース
52: $dbName = 'inventory';
53: // MySQL サーバ
54: $host = 'localhost:3306';
55: // MySQL の DSN 文字列
56: $dsn = "mysql:host={$host};dbname={$dbName};charset=utf8";
57:
58: //MySQL データベースに接続する
59: try {
60:     $pdo = new PDO($dsn, $user, $password); // データベースに接続します
61:     // プリペアドステートメントのエミュレーションを無効にする
62:     $pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
63:     // 例外がスローされる設定にする
64:     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
65: } catch (Exception $e) {
66:     $err = '<span class="error">エラーがありました。</span><br>';
67:     $err .= $e->getMessage();
68:     exit($err);
69: }
70:
71: try {
72:     // トランザクションを開始する
73:     $pdo->beginTransaction(); // トランザクション処理を開始します
74:     // SQL 文を作る
75:     $sql1 = "INSERT INTO goods (id, name, size, brand)
76:             VALUES (:id, :name, :size, :brand)";
77:     $sql2 = "INSERT INTO stock (goods_id, quantity) VALUES (:goods_id, :quantity)";
78:     // プリペアドステートメントを作る
79:     $insertGoods = $pdo->prepare($sql1);
80:     $insertStock = $pdo->prepare($sql2); // 商品レコード
81:     // プレースホルダに値をバインドする
82:     $insertGoods->bindValue(':id', $_POST["id"], PDO::PARAM_STR);
83:     $insertGoods->bindValue(':name', $_POST["name"], PDO::PARAM_STR);
84:     $insertGoods->bindValue(':size', $_POST["size"], PDO::PARAM_STR);
85:     $insertGoods->bindValue(':brand', $_POST["brand"], PDO::PARAM_STR);
86:     $insertStock->bindValue(':goods_id', $_POST["id"], PDO::PARAM_STR);
87:     $insertStock->bindValue(':quantity', $_POST["quantity"], PDO::PARAM_INT);
88:     // SQL 文を実行する
89:     $insertGoods->execute(); // 在庫レコード
90:     $insertStock->execute();
91:     // トランザクション処理を完了する
92:     $pdo->commit(); // トランザクション処理を終了します

```

2つの SQL 文を実行します


```

93:      // 結果報告
94:      echo " 商品データ／在庫データを追加しました。";
95:    } catch (Exception $e) {
96:      // エラーがあったならば元の状態に戻す   トランザクション処理を行っている最中にエラーが
97:      $pdo->rollBack();                          発生したならば、元の状態に戻します
98:      echo '<span class="error"> 登録エラーがありました。</span><br>';
99:      echo $e->getMessage();
100:    }
101:    ?>
102:    <hr>
103:    <p><a href="?php echo $gobackURL ?>">戻る </a></p>
104:  </div>
105: </body>
106: </html>

```

トランザクション処理を行う

トランザクション処理は、例外処理と組み合わせて行います。まず、`beginTransaction()` を実行してトランザクション処理を開始し、データベースの操作が完了したならば `commit()` を実行してトランザクション処理を終了します。

トランザクション処理を行っている間にエラーが発生すると例外がスローされて `catch` の構文が実行されます。ここで `rollBack()` を実行することで、トランザクション処理を行う前の状態に戻すことができます。

php 商品の追加に合わせて在庫も追加する（例外処理）

«sample» transaction/insert_goods.php

```

71:    try {
72:      // トランザクションを開始する
73:      $pdo->beginTransaction();

      ここでデータベースの操作を行う

91:      // トランザクション処理を完了する
92:      $pdo->commit();
95:    } catch (Exception $e) {
96:      // エラーがあったならば元の状態に戻す
97:      $pdo->rollBack();
100:    }

```

このプログラムでは新商品の追加だけができるようになっていて、登録済みの商品 ID の商品を追加するとエラーになって処理が中断し、トランザクション処理を開始する前の状態に戻ります。

1. 新しい商品を追加します

商品ID: TW11

商品名: カーディガン

サイズ: M

ブランド: アドレス

個数: 3

追加する

2. 商品と在庫のデータが追加されました

商品データ/在庫データを追加しました。

戻る

1. 使われている商品 ID で追加します

商品ID: A12

商品名: ドライソックス

サイズ: L

ブランド: ファインスカイ

個数: 5

追加する

2. エラー表示で中断し、追加処理前の状態に戻ります。

登録エラーがありました。

SQLSTATE[23000]: Integrity constraint violation: 1062 Duplicate entry 'A12' for key 'PRIMARY'

戻る